PED v3.0

SEN GREATS

Liam Ince 602330

Tetelo Phahladira 601950

Ofentse Vilakazi 603090

M3 Presentation Link:

https://belgiumcampusacza-my.sharepoint.com/:v:/g/personal/603090_student_belgiumcampus_ac_za/IQCfHcaOeXYhS5SCstyQu3niAdUEdvdlGy-FgAVwaw_O5oo?e=j43lYu

Contents

Project Overview: 5
Section A 5
Problem and Business Need 5
Stakeholder Analysis 7
Scope Baseline 9
Project Constraints 13
Assumptions 16
Requirements 20
Functional Requirements 20
Non Functional Requirements 21
Functional Acceptance Criteria 23
Non-Functional Acceptance Criteria 25
Requirements Prioritisation and Traceability Matrix 26
Requirements Prioritisation 26
1. Prioritisation Method 26
2. Priority Categories 26
3. Functional Requirements Prioritisation 26
4. Non-Functional Requirements Prioritisation 28
5. Important Prioritisation Decisions 29
M1-18: AI Usage Register 30
Section B 33
ASRs and quality drivers 33
Architecture decision and diagrams 37
Data and Persistence Baseline 42
ERD 42
Structure 43
Relationships 44
Access Patterns 44
Integrity 45
Consistency 45
Sensitivity 46
Expected Growth 46
Bottleneck / SPOF / Scalability / Availability / Backup 47
Persistence Model Decision 48
Selected M2 technology baseline 49
Why this stack fits CivicConnect 49
Current M2 status 49
Initial design decisions — informed by A2 research 49
Design Problem 1 – Excessive Dependencies between CivicConnect Components 49
Design Problem 2: Components Having Too Many or Unrelated Responsibilities 52
Initial API / integration decisions where applicable 54
(ADR-008) Decision 1: 54
(ADR-009) Decision 2: 55
Deployment compatibility 56
Current deployment direction 56
Configuration and security 57
Deployment risks and deferred decisions 57
SCM and CI adoption (M2-12) 57
Repository workflow 57
Initial GitHub Actions CI 58
Deferred after M2 58
Meaningful development and application evidence (M2-13) 58
Implemented in M2 58
Verification evidence 59
Current limitations / controlled future work 59

Current end-to-end trace 59
Section C: 64
M3 Controlled Construction, Verification and Release Evidence 64
C.1 Implementation status and quality evidence 64
C.2 Selected meaningful unit/component and integration tests 65
C.3 Manual black-box tests: five executed and passed 66
C.4 End-to-end, security and operational checks 66
C.5 Evidence photographs (selected excerpts) 67
C.6 Release-readiness judgement 70
C.7 Residual risk statement 71
C.8 RTM and controlled-evidence pointers 71
Section D: 72
Technical Debt Register: 73
Change Request Register: 74
References 75

Project Overview:

CivicConnect — A community service request management platform developed for the SEN381 Software Engineering project, focusing on requirements, traceability, quality, security, and controlled software engineering.

Section A

Problem and Business Need

CivicConnect addresses the problems caused by managing community service requests through disconnected channels such as email, telephone calls, WhatsApp, spreadsheets and paper records. This fragmented process can result in requests being duplicated, overlooked, incorrectly assigned or lost.

Requesters have limited visibility of their requests, staff struggle to prioritise and coordinate work, and management lacks reliable information for monitoring service performance and accountability (Belgium Campus ITversity, 2026 a).

The organisation therefore needs a controlled digital platform for submitting, managing, monitoring and reporting on service requests. CivicConnect will provide a single, traceable record of each request throughout its lifecycle. Its intended business value is to improve visibility, coordination, accountability and reporting without creating an unsustainable technical, operational or financial burden.

Stakeholder and Business-Value Link

Stakeholder	Current problem	Intended value
ST-001: Requesters	Limited confirmation, feedback and visibility of request progress	Clear confirmation, status tracking and meaningful feedback
ST-002: Service staff	Difficulty prioritising, assigning and coordinating requests	Better request organisation, ownership and controlled status management
ST-003: Management and oversight personnel	Limited reliable information about outstanding and completed work	Consistent information for monitoring, reporting and accountability
ST-004: Community-focused organisation	No single controlled record of the service-request lifecycle	A reliable and traceable process that improves overall service management

Stakeholder Analysis

The stakeholder analysis identifies the people and groups who use, manage, support or are affected by CivicConnect. Their needs and expectations provide the basis for defining the project scope and requirements. The main stakeholder groups were identified from the project scenario and minimum business capabilities in the Master Project Brief (Belgium Campus ITversity, 2026 a).

Stakeholder Register

ID	Stakeholder	Role and interest	Main needs and expectations	Influence	Interest
ST-001	Requesters	Submit service requests and monitor their progress.	A simple submission process, confirmation that requests were received, accurate status updates, meaningful feedback and access to previously submitted requests.	Medium	High
ST-002	Service staff	Receive, manage and resolve service requests.	Complete request information, clear priorities, useful search and filtering, controlled status updates and clear assignment of responsibility.	High	High
ST-003	Management and oversight personnel	Monitor service delivery, outstanding work and staff accountability.	Reliable service information, consistent reporting and visibility of open, overdue, resolved and closed requests.	High	High
ST-004	Community-focused organisation	Owns the platform and is responsible for improving the service-request process.	A reliable, secure and affordable platform that improves coordination and accountability without creating an unsustainable operational or financial burden.	High	High
ST-005	System administrators and IT support	Maintain the platform and support authorised access to its functions.	Secure access control, manageable maintenance, reliable operation and sufficient technical documentation.	High	Medium
ST-006	Lecturer	Evaluate and monitor project progression as well as give clearance at given milestones before next steps can be taken.	Accessibility and insight into project. Presentation of milestones. Expect high-quality work with a working deliverable product at the end. Can influence changes in project.	High	High

Requesters have high interest because they are directly affected by the submission and tracking process, but they have less authority over organisational decisions. Service staff and management have both high influence and high interest because they use the platform to manage requests and influence how the service process operates. The organisation has high influence because it owns the platform and is responsible for its cost and operation. System administrators have high technical influence, although they may have less interest in the daily service-request process.

Stakeholder Conflicts and Competing Expectations

Stakeholders involved	Conflict or competing expectation	Effect on the project
Requesters and service staff	Requesters may want a quick and simple submission process, while staff need enough information to categorise, prioritise and resolve requests correctly.	The submission process should remain easy to use while requiring the information needed to process a request.
Requesters and the organisation	Requesters may want complete visibility of request information, but some requests may contain sensitive information.	Access to request information must be restricted so that users only view information they are authorised to access.
Management and service staff	Management may require detailed records for reporting and accountability, while staff may find excessive data entry time-consuming.	The system should record important actions and status changes without creating unnecessary administrative work.
The organisation and system users	Users may want additional features, while the organisation must work within schedule, cost and team-capacity constraints.	Essential capabilities must be prioritised, while lower-priority features may need to be deferred to keep the project scope manageable.

Examples of Requirements Derived from Stakeholder Needs

Requirement ID	Stakeholder source	Identified need	Derived requirement
FR-001	ST-001: Requesters	Ability to submit a service request	The system shall allow an authenticated requester to submit a service request with the required information.
FR-002	ST-001: Requesters	Confirmation and visibility of request progress	The system shall provide a unique reference number after submission and allow the requester to view the current status of the request.
FR-003	ST-002: Service staff	Clear assignment of responsibility	The system shall allow authorised staff to assign or accept responsibility for a service request.

Requirement ID	Stakeholder source	Identified need	Derived requirement
FR-004	ST-003: Management and oversight personnel	Visibility of service activity	The system shall allow authorised management users to view open, overdue, resolved and closed requests.
NFR-001	ST-004 and ST-005	Protection of sensitive information	The system shall restrict access to functions and request information according to the authenticated user's assigned role.

These requirement examples establish the connection between stakeholder needs and Phase C. Their identifiers, priority, acceptance criteria and later traceability evidence must remain consistent in the requirements baseline and Requirements Traceability Matrix.

Scope Baseline

The CivicConnect scope baseline defines the capabilities that the team commits to delivering and establishes boundaries to prevent uncontrolled scope growth. It is based on the minimum business capabilities specified in the Master Project Brief and the needs of the stakeholders identified in M1-02. Any change to the approved scope must be assessed, recorded and approved through the project's change-control process (Belgium Campus ITversity, 2026 a).

In-Scope Capabilities

Scope ID	In-scope capability	Primary stakeholder
SC-001	Allow requesters to submit new service requests with the required information.	ST-001: Requesters
SC-002	Allow requesters to select a request category using controlled category options.	ST-001: Requesters
SC-003	Provide confirmation and a unique reference number when a request is successfully submitted.	ST-001: Requesters
SC-004	Allow requesters to view the current status of their submitted requests.	ST-001: Requesters
SC-005	Allow requesters to view a history of their previously submitted requests.	ST-001: Requesters
SC-006	Provide meaningful feedback when a request is accepted, rejected, updated, resolved or closed.	ST-001: Requesters
SC-007	Allow authorised staff to view service requests relevant to them.	ST-002: Service staff
SC-008	Allow authorised staff to search, filter and sort requests using relevant criteria.	ST-002: Service staff
SC-009	Allow authorised staff to view complete service-request details.	ST-002: Service staff

Scope ID	In-scope capability	Primary stakeholder
SC-010	Allow authorised staff to assign or accept responsibility for a request.	ST-002: Service staff
SC-011	Allow authorised staff to update request statuses through controlled status transitions.	ST-002: Service staff
SC-012	Allow authorised staff to record actions, comments and resolution information.	ST-002: Service staff
SC-013	Allow authorised staff to resolve or close service requests.	ST-002: Service staff
SC-014	Allow authorised management users to view open, overdue, resolved and closed requests.	ST-003: Management and oversight personnel
SC-015	Allow authorised management users to view service information according to category, status and other justified measures.	ST-003: Management and oversight personnel
SC-016	Maintain a traceable record of important actions and status changes throughout the lifecycle of a service request.	ST-002, ST-003 and ST-004
SC-017	Control access to system functions and request information according to the user’s authorised role.	ST-004 and ST-005

The feedback included in the current scope may be provided within CivicConnect. Integration with external communication services is not required for the initial project baseline.

System Boundary

CivicConnect will allow users to submit, categorise, assign, track and manage community service requests. It will also allow management to monitor requests and view reports. The system will only manage the request and its information; it will not carry out the actual service or repair. Emergency services, unrelated organisational processes and features deferred to a later stage are outside the current system boundary.

Out-of-Scope Items

Scope ID	Out-of-scope item	Reason
OS-001	Performing the physical maintenance, repair, security or support work described in a request	CivicConnect manages information and the request lifecycle; it does not perform the service itself.
OS-002	Payroll, employee-performance and human-resource management	These activities are unrelated to the service-request problem defined in the project brief.
OS-003	Inventory, procurement and payment processing	The project focuses on submitting, managing, monitoring and reporting on service requests rather than managing organisational finances or purchasing.
OS-004	Replacing emergency-response services	CivicConnect may record security-related concerns, but it is not intended to replace police, medical or other emergency-response channels.
OS-005	Replacing the organisation's unrelated administrative systems	The project boundary is limited to community service-request management.

Deferred or Future Scope

Scope ID	Deferred capability	Reason for deferment
FS-001	A separate native Android or iOS application	A native application is not required by the minimum business capabilities and would introduce additional development, compatibility and testing work.
FS-002	WhatsApp, SMS and external email-service integration	These integrations introduce third-party dependencies, operational costs, security concerns and additional testing requirements. Comparable organizations have taken a similarly phased path. New Haven integrated SeeClickFix into its existing request-handling workflow incrementally, alongside its original reporting channel rather than as a single, combined rollout (SeeClickFix, n.d.).
FS-003	Integration with external municipal or organisational systems	The required external systems, interfaces, permissions and compatibility information have not yet been established.

Scope ID	Deferred capability	Reason for deferment
FS-004	Artificial intelligence-based request categorisation or prioritisation	The minimum capability can be met through controlled categories and staff prioritisation. AI would introduce additional accuracy, transparency and verification concerns.
FS-005	Advanced predictive reporting	Milestone 1 requires useful service information, but predictive analysis is not part of the minimum business capabilities.
FS-006	Interactive maps and live location tracking	This would introduce additional data, privacy, interface and integration requirements that are not essential to the core request-management process.
FS-007	Multilingual support	This may improve future accessibility but requires confirmed language needs, translation resources and additional interface and testing work.

Deliberate Deferment Decision

The team has deliberately deferred a separate native mobile application from the current project scope. The Master Project Brief requires the core service-request capabilities but does not require a native mobile application. Building and testing separate mobile applications would increase the workload for a three-person team and could reduce the time available for completing and verifying the mandatory capabilities. The team will therefore prioritise a complete, controlled and testable core platform. A native mobile application may be reconsidered later if stakeholder evidence shows that it is necessary and the available schedule and resources can support it. This approach matches other comparable platforms. FixMyStreet, a widely used civic-reporting system, has operated successfully as a primarily web-based platform across several countries without needing a native mobile app as a condition for adoption (Peter Matthews, 2018).

External messaging integrations have also been deferred. CivicConnect will still provide meaningful request feedback within the platform, while WhatsApp, SMS or external email notifications can be considered later after their cost, security, privacy and integration implications have been investigated.

Major Scope Dependencies

The scope depends on the organisation confirming the request categories, request statuses and user roles. It also depends on stakeholders being available to confirm the requirements, staff keeping request information updated and the selected low-cost services supporting the expected number of users and requests. These dependencies are recorded in ASM-002 to ASM-008. If any of them change, the team must check how the scope and related requirements are affected and record the change.

Scope Control

This scope statement forms the proposed M1 baseline. Once it has been reviewed and signed off by the team, items may not be added, removed or changed informally. Any proposed change must identify the affected requirements and consider its effect on schedule, cost, resources, quality, security, risk, testing and later project artefacts, as required by the Milestone 1 scope-control standard (Belgium Campus ITversity, 2026 b).

Project Constraints

Project constraints are conditions that limit or influence how CivicConnect can be planned, developed, tested and operated. These constraints must influence the project's scope, requirements and later engineering decisions rather than only being listed. The following constraints are based on the Master Project Brief and the Milestone 1 objective (Belgium Campus ITversity, 2026 a) (Belgium Campus ITversity, 2026 b).

Constraint Register

The constraints are grouped into scope, technical, schedule, resource, quality, security and privacy, and academic and project-governance categories.

ID	Category	Constraint	Engineering implication
CON-001	Scope and project	The agreed project scope must be baselined and controlled. Uncontrolled scope growth is not acceptable.	The team must prioritise the minimum business capabilities and avoid adding features without analysing their effect on requirements, schedule, cost, resources, quality, security, testing and risk. Approved scope changes must be recorded and reflected in the affected project artefacts.
CON-002	Schedule	The project must progress through four formal milestones within the SEN381 delivery period.	Work must be planned according to the milestone deadlines, and evidence must be produced progressively. Delays or additional features may reduce the time available for implementation, testing, security review and documentation.
CON-003	Resource	CivicConnect must be completed by a team of exactly three students, and each member must provide evidence of an individual contribution.	The project scope and division of work must remain realistic for three students. Responsibilities should be assigned clearly, while important artefacts must still be understood and reviewed by the entire team.
CON-004	Resource and cost	The team should use free or low-cost development, hosting and supporting services where practical. Limitations and likely costs outside the educational environment must still be identified.	Technology and deployment options must be evaluated according to their free-tier limits, availability and possible future operational costs. A service should not be selected only because it is free if its limitations create unacceptable security, reliability or deployment risks.
CON-005	Quality	Quality expectations must be defined and later demonstrated using measurable evidence.	Phase C must include measurable non-functional requirements and acceptance criteria. Later milestones must provide appropriate testing and quality evidence rather than claiming that the platform is reliable or usable without proof.

ID	Category	Constraint	Engineering implication
CON-006	Security and Privacy	Security and privacy must be considered throughout the project lifecycle and cannot be treated as a final addition.	The team must define security requirements, restrict access according to authorised roles, protect sensitive request information and follow secure repository, development and deployment practices. Security risks and remaining limitations must also be recorded.
CON-007	Technical	No technology stack, architecture or platform is prescribed. Any selection must be justified against the requirements, constraints and actual capability of the team.	The final technology decision must be deferred until suitable alternatives have been investigated. The team must consider security, maintainability, testing support, compatibility, deployment, cost and the learning curve before committing to a stack.
CON-008	Academic and project governance	Project artefacts and changes must be controlled through the team repository using the required issues, branches, meaningful commits, pull requests and reviews.	Team members must create authentic evidence while the work is being completed. Documentation and code cannot simply be uploaded in bulk before the assessment, and changes to baselined artefacts must follow the agreed control process.

Constraint Interactions, Trade-offs and Ripple Effects

The constraints are connected, meaning that a change in one area may create consequences in several other areas.

Related constraints	Interaction or trade-off	Possible ripple effect
Team resources, schedule and scope	A three-person team has limited capacity within the milestone deadlines.	Increasing the scope creates additional requirements, development and testing work. This may delay delivery or reduce the time available for quality and security activities.
Cost, technology and schedule	A free service may reduce immediate cost but may have usage, compatibility or deployment limitations.	If the limitations are discovered late, the team may need to change platforms, update technical decisions and repeat implementation or testing work.
Security, usability and quality	Strong access controls help protect sensitive information but may make the system more difficult to use if they are poorly designed.	The team must balance secure access with a clear user experience and later test both security behaviour and usability-related requirements.

Related constraints	Interaction or trade-off	Possible ripple effect
Scope control and traceability	Changing a baselined feature affects more than the scope statement.	The related stakeholder needs, requirements, acceptance criteria, Requirements Traceability Matrix, risks, design, issues and tests may also need to be updated.
Schedule and quality	Schedule pressure may encourage the team to reduce testing or documentation.	This could allow defects or security weaknesses to remain undetected and would weaken the evidence required in later milestones.

Key Constraint Interaction

The most significant current interaction is between team capacity, schedule and scope. CivicConnect must be completed by three students within four formal milestones. If the team adds non-essential features, the amount of requirement, implementation, security and testing work will increase. This could place milestone deadlines and overall system quality at risk. The team has therefore limited the baseline to the required business capabilities and deferred additional features such as native mobile applications, external messaging integrations and artificial intelligence-based request prioritisation.

The technology constraint also creates an important trade-off. Although free or low-cost technologies may reduce immediate project costs, an unfamiliar platform may introduce a learning curve and increase the risk of delays or implementation errors. Technology choices must therefore consider both financial cost and the team's ability to use, test and maintain the selected solution.

Assumptions

Assumptions are conditions that the team currently believes to be true but has not yet fully confirmed. They differ from constraints because constraints are known limitations, while assumptions still require verification. If an important assumption is incorrect, it may affect the project's scope, requirements, design, schedule, cost, quality or security. Material assumptions must therefore be monitored and considered in the Risk Register (Belgium Campus ITversity, 2026 a).

The assumptions cover stakeholders, users, technical matters, operations and external services. CivicConnect is not currently expected to depend on external datasets or third-party data feeds because request information will be entered and managed within the system. If an external data source is introduced later, it will need to be validated and assessed through change control.

Assumptions Register

ID	Assumption	Why it matters	Impact if incorrect	Verification action	Owner	Status
ASM-001	Intended users will have access to a suitable device, supported browser and sufficiently reliable internet connection when using CivicConnect.	Users must be able to access the platform to submit or manage requests.	The team may need to reconsider device compatibility, performance requirements, accessibility or possible offline support.	Confirm the expected user environment and later test the platform on representative devices, browsers and network conditions.	Requirements lead	To be verified
ASM-002	The organisation will be able to provide and approve a controlled list of service-request categories.	Requesters must categorise requests, while staff need consistent categories for filtering and reporting.	Incomplete categories could cause requests to be misclassified and make reports unreliable.	Propose an initial category list based on the request types in the brief and obtain approval before implementation.	Stakeholder and scope lead	To be verified

ID	Assumption	Why it matters	Impact if incorrect	Verification action	Owner	Status
ASM-003	The organisation will be able to define the allowed request statuses and valid transitions between them.	CivicConnect must provide controlled status updates throughout the request lifecycle.	Unclear status rules could result in inconsistent updates, incorrect reporting and requirements that cannot be tested properly.	Develop a proposed request lifecycle and confirm the meaning and order of each status before detailed design.	Requirements lead	To be verified
ASM-004	Requester, staff, management and administrative responsibilities can be separated through clearly defined user roles.	Role definitions are needed to control access to functions and sensitive request information.	Users could receive too much or too little access, creating security, privacy and usability problems.	Define a role-permission matrix and review it against the approved requirements before implementation.	Requirements and security lead	To be verified
ASM-005	Service staff will update request ownership, actions and statuses consistently while processing requests.	Tracking, reporting and accountability depend on staff keeping request information current.	Requesters and management could receive outdated or misleading information even if the software functions correctly.	Define required workflow actions and later verify them through acceptance criteria and user testing.	Stakeholder and requirements lead	To be verified
ASM-006	Suitable stakeholder representatives will be available to clarify needs and review the team’s interpretation of the CivicConnect process.	Stakeholder validation helps confirm that the scope and requirements address the intended problem.	Incorrect interpretations may remain undetected and lead to unsuitable requirements or late changes.	Record unresolved questions and obtain feedback through the lecturer, project representative or approved stakeholder-validation process.	Project team	To be verified

ID	Assumption	Why it matters	Impact if incorrect	Verification action	Owner	Status
ASM-007	The expected number of users, requests and stored records will remain within the capabilities of the platform and services selected later.	Expected usage affects performance, storage, cost, architecture and deployment decisions.	Higher-than-expected usage could cause poor performance, additional cost or the need to change technology.	Establish reasonable usage estimates and evaluate them when comparing technology and deployment alternatives.	Technology and deployment lead	To be verified
ASM-008	Free or low-cost development and deployment services will remain available and sufficient during the project period.	The project is expected to prefer affordable services, but these services may have usage or feature limitations.	Pricing, availability or free-tier changes could require a new platform decision and cause delays.	Research current service limits before selection and record suitable alternatives.	Technology and deployment lead	To be verified
ASM-009	All three team members will have access to the agreed repository, development tools and compatible working environments.	Controlled collaboration and individual contribution evidence depend on each member being able to use the project tools.	Access or compatibility problems could delay work, reduce contribution evidence or create integration difficulties.	Confirm repository access and tool compatibility for all team members before implementation begins.	GitHub and governance lead	To be verified

Assumption Analysis and Risk Treatment

The assumptions with the greatest current effect are ASM-004, ASM-005, ASM-006 and ASM-008. Unclear user roles could create unauthorised access, while inconsistent staff updates could prevent CivicConnect from providing reliable tracking and reporting. A lack of stakeholder validation could result in incorrect scope or requirements, and the loss or limitation of a selected low-cost service could affect the schedule and deployment plan.

These assumptions must be considered when Phase D develops the Risk Register. If an assumption is confirmed, its status and supporting evidence should be updated. If an assumption proves incorrect, the team must assess its effect on the scope, requirements, constraints, risks and related engineering decisions instead of silently changing the baseline.

Requirements

Review Status

Section	Current status	Next action
Functional requirements	Based on the requirements received from the team, with the confirmed reference number and in-app notification clarifications included.	Confirm wording, categories, roles, and workflow states.
Non-functional requirements	Draft targets written without selecting a technology stack.	Confirm measurable targets and constraints.
Acceptance criteria	Draft criteria linked to every FR and NFR.	Review each criterion for correctness and testability.

Functional Requirements

The following requirements describe what CivicConnect must do. The IDs should remain stable once the team approves the baseline.

ID and title	Functional requirement	Business requirement/need addressed
FR-001 Submit Service Request	The system shall allow a requester to submit a new service request by providing the required information. After successful submission, the system shall generate and display a unique reference number.	Creates one controlled method for recording requests, reduces lost requests and gives each request a traceable identifier
FR-002 Categorise Service Request	The system shall allow requesters to select a request category from a predefined list.	Supports correct routing, prioritisation and reporting.
FR-003 View Request Status	The system shall allow requesters to view the current status of their submitted requests.	Improves transparency and reduces uncertainty about progress.
FR-004 View Request History	The system shall allow requesters to view their previously submitted requests.	Provides a traceable history and supports accountability.
FR-005 Receive In-App Request Notifications	The system shall provide notifications within CivicConnect when a request is accepted, rejected, updated, resolved or closed.	Keeps users informed without relying on the deferred SMS, WhatsApp or email integrations.

ID and title	Functional requirement	Business requirement/need addressed
FR-006 View Assigned Requests	The system shall allow authorised staff to view requests relevant to their role or assigned responsibilities.	Improves coordination and supports role-based responsibility.
FR-007 Search Filter and Sort Requests	The system shall allow authorised staff to search, filter and sort service requests.	Helps staff organise workloads and find urgent or overdue requests.
FR-008 View Detailed Request Information	The system shall allow authorised staff to view the complete recorded details of a selected request.	Supports informed decisions and reduces errors caused by missing information.
FR-009 Assign Responsibility	The system shall allow authorised staff or supervisors to assign requests to authorised personnel.	Clarifies ownership and improves accountability.
FR-010 Update Request Status	The sys tem shall allow authorised staff to update a request through the approved workflow stages.	Provides visibility of progress and creates a lifecycle record.
FR-011 Record Actions and Comments	The system shall allow authorised staff to record comments, actions taken and progress notes against a request.	Supports teamwork, handovers and an auditable history.
FR-012 Resolve and Close Requests	The system shall allow authorised staff to mark completed requests as resolved and close them according to the approved workflow.	Supports accurate completion tracking and reporting.
FR-013 View Service Activity Dashboard	The system shall provide management users with a dashboard showing service activity information.	Supports oversight and decision-making.
FR-014 Monitor Requests	The system shall allow management users to monitor open, overdue, resolved and closed requests.	Helps identify bottlenecks and supports performance management.
FR-015 Generate Reports and Statistics	The system shall provide reports and statistics based on request category, status, resolution rate and workload.	Supports reporting, analysis and evidence-based decisions.

Non Functional Requirements

ID and category	non-functional requirement	Business requirement/need addressed
NFR-001 Performance	Under the agreed expected load, the main user actions should complete within three seconds for at least 95% of requests.	Supports timely service delivery by reducing delays when users submit requests or staff access request information.
NFR-002 Security and access control	CivicConnect shall restrict functions and request information according to approved user roles and permissions	Protects accountability and appropriate access by ensuring users can only perform functions and view request information relevant to their role.
NFR-003 Privacy	CivicConnect shall collect only information needed to manage service requests and shall prevent unauthorised access to personal or sensitive information.	Protects requester information by limiting collection and access to information necessary for managing service requests.
NFR-004 Data integrity	CivicConnect shall preserve valid request data and prevent invalid status changes or incomplete required records.	Supports accurate service management and reporting by ensuring request records remain complete and valid throughout their lifecycle.
NFR-005 Reliability and recovery	CivicConnect shall not lose a successfully saved request when a user refreshes the page or signs in again, and recoverable failures shall show a clear message.	Maintains continuity of service and trust by preventing loss of successfully submitted requests and providing clear feedback when recoverable failures occur.
NFR-006 Usability	A first-time user shall be able to submit and track a request using clear labels, instructions, and feedback without specialist training.	Makes CivicConnect practical for the intended community by allowing first-time users to submit and track requests without specialist training.
NFR-007 Accessibility	Core request submission and tracking functions shall support keyboard navigation, labelled controls and readable colour contrast.	Ensures the service can be used by users with different accessibility needs, supporting inclusive access to core request submission and tracking.
NFR-008 Compatibility	CivicConnect shall work on the browsers and device sizes approved for the intended users.	Ensures users can access CivicConnect through the approved devices and browsers used to interact with the service.

ID and category	non-functional requirement	Business requirement/need addressed
NFR-009 Auditability	CivicConnect shall record important request-lifecycle changes, including the action, date and responsible user where applicable.	Supports accountability and transparency by retaining evidence of important request-lifecycle actions and who performed them.
NFR-010 Maintainability	The solution shall use a documented, modular structure so that authorised team members can test and change one area without unnecessary changes to unrelated areas.	Supports sustainable project delivery and future change by keeping the solution understandable, structured and maintainable by authorised team members.
NFR-011 Scalability and capacity	The selected solution shall support the agreed expected number of users, requests and stored records without redesigning the core request process.	Ensures CivicConnect can continue supporting the expected level of service demand as the number of users, requests and stored records increases.
NFR-012 Cost	Development and deployment shall remain within the approved project budget, with free or low-cost service limits checked before selection.	Keeps the solution financially feasible within the student-project constraints and avoids selecting services whose usage costs could make the solution impractical.

Functional Acceptance Criteria

Each criterion is linked to one functional requirement and states an observable result that can later support testing and traceability.

Acceptance ID	Requirement	Acceptance criterion
AC-FR-001	FR-001	Given a requester has entered all required information, when the request is submitted successfully, then it is saved, and a unique reference number is displayed.
AC-FR-002	FR-002	Given the approved category list is available, when a requester creates a request, then one category can be selected and saved with the request.
AC-FR-003	FR-003	Given a requester has submitted a request, when they open it, then its current approved status is displayed.

Acceptance ID	Requirement	Acceptance criterion
AC-FR-004	FR-004	Given a requester has previous requests, when they open their history, then those requests and their reference numbers are shown.
AC-FR-005	FR-005	Given a tracked request changes to an approved notification-triggering state, when the update is saved, then the relevant user can see an in-app notification.
AC-FR-006	FR-006	Given an authorised staff member signs in, when they open their work list, then only requests available to their role or assignment are shown.
AC-FR-007	FR-007	Given requests exist, when authorised staff apply a search term, filter or sort order, then the displayed results match the chosen condition.
AC-FR-008	FR-008	Given authorised staff select a request, when its details are opened, then all information recorded for that request is displayed.
AC-FR-009	FR-009	Given an unassigned or reassignable request exists, when an authorised user selects eligible personnel and confirms, then the responsible person is recorded.
AC-FR-010	FR-010	Given a request is in a valid current state, when authorised staff select an allowed next status, then the new status and update time are recorded.
AC-FR-011	FR-011	Given authorised staff are viewing a request, when they save a comment, action or progress note, then it appears in the request history with the responsible user and time.
AC-FR-012	FR-012	Given the approved completion conditions are met, when authorised staff resolve or close the request, then its final state and completion information are recorded.
AC-FR-013	FR-013	Given service-request data exists, when a management user opens the dashboard, then the agreed activity measures are displayed using current stored data.
AC-FR-014	FR-014	Given requests have different statuses and due conditions, when a management user selects a monitoring view, then open, overdue, resolved and closed requests can be identified.
AC-FR-015	FR-015	Given service-request data exists, when a management user requests a report, then statistics are calculated using the selected category, status, resolution-rate or workload information.

Alignment note: Acceptance criteria apply to both functional and non-functional requirements. The criteria below therefore provide observable/testable evidence for each NFR, just as the functional acceptance criteria do for each FR.

Non-Functional Acceptance Criteria

These criteria provide an initial way to verify the non-functional requirements. Criteria containing an agreed or approved value must be updated after the team confirms that value.

Acceptance ID	Requirement	Acceptance criterion
AC-NFR-001	NFR-001	A performance test under the agreed expected load shows that at least 95% of the selected main actions complete within three seconds.
AC-NFR-002	NFR-002	Role-based access tests confirm that every tested role can access its permitted functions and cannot access restricted functions or information.
AC-NFR-003	NFR-003	A review of forms, permissions and stored fields confirms that only approved information is collected and unauthorised roles cannot view protected information.
AC-NFR-004	NFR-004	Validation tests reject incomplete required records and status changes that are not allowed by the approved workflow.
AC-NFR-005	NFR-005	A successfully saved request remains available after refresh and a simulated recoverable failure gives the user a clear message without creating an unintended duplicate.
AC-NFR-006	NFR-006	Representative first-time users can complete the agreed submission and tracking walkthrough, and any blocking usability problems are recorded and corrected or accepted.
AC-NFR-007	NFR-007	The agreed core screens can be operated by keyboard, controls have understandable labels and the selected contrast check passes.
AC-NFR-008	NFR-008	The agreed core workflows pass on every browser and screen size listed in the approved compatibility list.
AC-NFR-009	NFR-009	Tests confirm that each approved lifecycle action records the action, date and responsible user where applicable.
AC-NFR-010	NFR-010	The project structure and developer documentation are reviewed against the team standards, and automated checks pass for the changed area.
AC-NFR-011	NFR-011	A capacity test using the agreed volume estimates completes without loss of core functionality or failure of the request process.

Requirements Prioritisation and Traceability Matrix

This document is currently based on the draft functional requirements. It will be updated once the non-functional requirements and acceptance criteria have been completed and reviewed.

Requirements Prioritisation

1. Prioritisation Method

The MoSCoW method was used to prioritise the CivicConnect requirements.

2. Priority Categories

Priority	Meaning
Must Have	Essential for the system to meet its main purpose or operate safely and correctly
Should Have	Important but not essential for the first usable version
Could Have	Desirable and valuable but can be postponed without preventing the system from operating
Won't Have for now	Excluded from the current release but may be reconsidered later

3. Functional Requirements Prioritisation

Requirement ID	Requirement Summary	Priority	Justification
FR-001	Submit Service Request	Must	The platform cannot operate without capturing requests and providing a unique reference number.
FR-002	Categorise Service Request	Must	Categories are needed to organise and route requests correctly
FR-003	View Request Status	Must	Request visibility is one of the main problems CivicConnect must solve.
FR-004	View Request History	Should	It improves traceability, but the core request process can initially operate without a complete history view.
FR-005	Receive Request Notifications	Should	In-platform notifications improve communication, but users can still check their request status directly.
FR-006	View Assigned Requests	Must	Staff need access to relevant requests before they can process them.

Requirement ID	Requirement Summary	Priority	Justification
FR-007	Search, Filter and Sort Requests	Should	It improves staff efficiency but is not essential for processing a small initial number of requests.
FR-008	View Detailed Request Information	Must	Staff need complete information to process requests correctly.
FR-009	Assign Responsibility	Must	Clear ownership is required for accountability and prevents requests from being overlooked.
FR-010	Update Request Status	Must	Status updates are required to track each request through its lifecycle.
FR-011	Record Actions and Comments	Must	Actions and notes create the traceable record needed for coordination and accountability.
FR-012	Resolve and Close Requests	Must	The request lifecycle must include a controlled way to record completion.
FR-013	View Service Activity Dashboard	Could	A dashboard is useful, but management can initially access the underlying information through monitoring and reports.
FR-014	Monitor Open and Overdue Requests	Must	Management needs to identify outstanding work and service-delivery problems.
FR-015	Generate Reporting and Analysis Information	Must	Reporting supports the stated business need for monitoring and accountability.

No baseline functional requirement was classified as Won’t-have because every listed requirement supports the approved CivicConnect scope. Features already excluded or deferred, including external messaging integrations, a native mobile application and AI-based prioritisation, are recorded separately in the Phase B scope baseline.

4. Non-Functional Requirements Prioritisation

Requirement ID	Requirement Summary	Priority	Justification
Pending	M1-05 Non-Functional Requirements	Not yet assigned	To be completed after the non-functional requirements have been finalised and reviewed.

5. Important Prioritisation Decisions

The most important prioritisation decisions were based on whether a requirement is needed to complete and control the core service-request lifecycle.

FR-001, FR-002, FR-003 and FR-006 to FR-012 were classified as Must Have because they allow a request to be submitted, organised, assigned, processed, tracked and closed. Without these capabilities, CivicConnect would not provide a complete and traceable request-management process.

FR-014 and FR-015 were also classified as Must Have because monitoring and reporting directly support the business need for visibility and accountability.

FR-004, FR-005 and FR-007 were classified as Should Have because they improve transparency and staff efficiency, but the first workable version can still process requests without them. FR-013 was classified as Could Have because the dashboard presents information that can initially be obtained through the monitoring and reporting functions.

M1-18: AI Usage Register

Date	Student	Tool	Engineering Task	AI Contribution	Verification	Decision	Issues Found
09/09/2026	Liam Axel Ince	Claude	Tighten and review language and grammar in registers M1-11 to M1-13	Suggested sentence reworks, identified redundancy.	Checked that the reworks matched my original intention of what I wanted to say and picked the things I thought relevant to rephrase using the example of better language and more summarized.	Chose certain revisions I agreed with that matched my intention and criteria and reworked into by myself into the register.	AI didn't understand the relation between the The ASM-001 and the FS-001 that I wanted to mention in R-008
09/09/2026	Tetelo Phahladira	Chatgpt	Functional and Non-functional requirements Acceptance criteria	AI assistance was used to review and align the existing NFR business-need column and to check the relationship between requirements and acceptance	The resulting content was reviewed against the existing working requirements and the project brief before inclusion.	The format was adopted. It clearly structured and clearly depicts how we intended to show the requirements addressed to the needs, as well	Non

				criteria against the project brief.		as depicting how which specifically.	
09/09/2026	Tetelo Phahladira	Chatgpt	GitHub issues	Ai was used to make the issue-tasks based of the project briefs. It also assisted us to make git bulk scripts to have the issues created all at once instead of one-by-one via the the GitHub Ui.	We continually checked the Milestone 1 document output and what the PED was specified to contain.	After validating, we ran the scripts and output was adopted.	The AI tends to deviate from scope. Hence validation against what is said in the briefs was pivotal
9/09/2026	Ofentse Vilakazi	ChatGPT	Phase B documentation and review	AI assisted with improving the wording, formatting tables and identifying areas that needed clearer explanations or supporting evidence.	I reviewed every Phase B section against the Master Project Brief, Milestone 1 requirements, GitHub issue tasks and acceptance criteria.	I used suggestions that improved clarity and structure.	Missing citations and evidence were identified. The system boundary, major scope dependencies, constraint categories and assumption impacts also needed clearer explanations.

09/09/2026	Ofentse Vilakazi	ChatGPT	GitHub branch, commit, pull-request and project-board workflow	AI provided step-by-step guidance for creating and switching branches, committing and pushing documents, updating pull requests and changing issue statuses.	I checked the branch name and git status output before following each command. I also confirmed the results on GitHub after pushing or updating a pull request.	I followed the commands only after confirming that they matched the correct branch and file. GitHub remained the source of truth for the final branch, commit and review status.	Branch switching was blocked by uncommitted changes, one PR was still a draft, and the repository project initially appeared unlinked from the Issues project view.
09/09/2026	Ofentse Vilakazi	ChatGPT	M1-07 Prioritisation and M1-08 Requirements Traceability Matrix	AI assisted with identifying possible links between Phase B sources, requirements and acceptance criteria.	I checked the requirement ID against the final requirements document and compared the RTM links with the Phase B stakeholder, scope, constraint and assumption tables.	I used the priorities and links that were consistent with the project baseline and corrected or clarified them where necessary. Design, test and release references were left as controlled placeholders for later milestones.	The unique reference number was initially missing from the functional requirements, notification wording was unclear, and design, test and release artefacts do not exist yet. These were clarified or recorded for later traceability.

Section B

ASRs and quality drivers

Architecturally Significant Requirements (ASRs) are requirements that are significant in designing a system's architecture and are therefore given a higher priority (Harikrishnan, 2024). This means that a system architect's decisions are influenced by these requirements. Bode's (2011) doctoral thesis on quality-goal-oriented architectural design frames this precisely: architectural analysis takes functional and non-functional requirements, together with the business-goal context, and from these "defines the architecturally significant requirements". These requirements then drive architectural synthesis—the creation of candidate solutions—and architectural evaluation, which determines whether the appropriate architectural decisions have been made. A **quality driver** is the more specific mechanism through which an ASR exerts that influence.

Quality Driver	**CivicConnect Evidence**	**Measure/Test**	**Architecture Implication**	**Grounding**
Performance	Functional Requirement NFR-001 (Non-Functional Requirement)	95% of selected actions ≤ 3 seconds	Influences application and data architecture	Silva et al. (2023) note that architecture-only performance metrics allow this type of response-time target to be evaluated against a design before code exists, rather than only after implementation.
Security	NFR-002/NFR-003	Role-based access tests	Requires authentication and authorisation boundaries	Sohofi & Dehbashi Dezfouli's (2025) review shows that Role-Based Access Control (RBAC) is a standard mechanism for "streamlining user management in complex

				organisational structures". For CivicConnect, this means that the architecture must expose an explicit authorisation boundary rather than embedding access checks ad hoc across components.
Data Integrity	NFR-004	Invalid records/status transitions rejected	Influences data model and validation	Consistent with the layered-enforcement findings already established for CivicConnect's persistence design, which includes client, application/service, and database-constraint layers.
Reliability	NFR-005	Saved request survives refresh/recoverable failure handled	Influences persistence and error handling	This is an architecturally significant requirement in Bode's sense because it forces a structural decision regarding transaction boundaries and

				rollback behaviour rather than a cosmetic one.
Maintainability	NFR-010	Modular structure + automated checks	Influences component/module boundaries	Silva et al. (2023) catalogue maintainability metrics precisely because it is one of the quality attributes most commonly evaluated at the architecture-only level, before full implementation.
Scalability	NFR-011	Capacity test against agreed volume	Influences architecture and technology choices	Losavio et al. (2004) show that scalability-related characteristics must be specified using explicit attributes, units, and scale types under the International Organization for Standardization (ISO) 9126-1 quality model. This allows key architectural options to be compared against a target volume rather

				than being assumed (p. 1).
Auditability	NFR-009	Lifecycle actions record user/date/action	Influences data model and service design	This is an architecturally significant requirement because it requires every state-changing service to emit a traceable record, rather than relying solely on the component that owns the requirement.
Cost	NFR-012 / CON-004 (Constraint)	Service limits/cost checked	Influences technology/deployment selection	This is directly linked to a project constraint (CON-004) rather than a stakeholder wish, which is precisely the type of traceable source that an ASR requires each driver to have.

Measurable or testable expectations

Every row already contains a stated **Measure/Test**. This is important because, according to Silva et al. (2023), architecture-level evaluation can be divided into scenario-based methods, such as the Architecture Tradeoff Analysis Method (ATAM), and metric-based methods. CivicConnect's table follows the metric-based approach, with one testable expectation associated with each driver. This allows the drivers to be evaluated at design time rather than being left as vague intentions.

Losavio et al. (2004) reinforce that a scalability or performance target is only useful if it is expressed using an explicit attribute, unit, and scale.

How drivers influence architecture, data, technology, and design

- **Architecture:** Security (NFR-002/003) requires an explicit authorisation boundary. Sohofi & Dehbashi Dezfouli's (2025) comparison of Discretionary Access Control (DAC), Mandatory Access Control (MAC), Role-Based Access Control (RBAC), and Attribute-Based Access

Control (ABAC) supports choosing RBAC specifically for CivicConnect, since its roles (requester, staff, administrator) map onto RBAC's organisational-role model rather than ABAC's finer-grained attribute logic.

- **Data:** Data Integrity (NFR-004) and Auditability (NFR-009) both force every state-changing service to validate transitions and emit a lifecycle record- a data-model consequence, not just a validation-logic one.
- **Technology:** Scalability (NFR-011) and Cost (NFR-012/CON-004) jointly constrain technology selection- a capacity target only means something once compared against a specific hosting tier's limits and cost.
- **Design:** Maintainability (NFR-010) forces component/module boundaries to be drawn deliberately, consistent with Silva et al.'s (2023) observation that maintainability is one of the quality attributes most commonly evaluated directly from the architecture description, before implementation exists.

Architecture decision and diagrams

Modern software systems depend more and more on organised architectural principles to control complexity, facilitate reuse, and direct quality-driven design (Hassouna, 2025). Software architecture is a high-level abstraction of a s system that depicts system structure (HAJIPOOR, et al., 2025).

Layered Monolithic Architecture

An architecture that allows for a single deployable application organised into a small set of horizontal layers with clear responsibilities (Hardzeenka, n.d.). The code is grouped into layers, namely the presentation layer (UI), application/service layer (Business logic), and Data Access Layer (Alvares, 2025).

1. Team capability

PED document has CON-003, which restricts development to exactly three development members and explicitly requires the scope and division of work to remain realistic for that team. This architecture would provide the team of this size with logical modularity without forcing distributed-system operations onto the project.

2. Deployment

SEN Greats report (Assignment 2) defines an in-process approach as components running in the same application and being built and deployed together as one unit. Constraint CON-004 in the PED document outlines that free or low-cost development, hosting, and supporting services should be preferred where practical.

A trade-off that might be experienced with a single deployment unit is that a change to one component means redeploying the entire application.

3. Complexity

NFR-010, documented in the PED, explicitly states that the system should remain modular and allow one area to change without unnecessary changes to unrelated areas. Hermanova (2024) mentions in their report that one purpose of software architecture is controlling and mitigating conceptual complexity and specifically identifies layered architecture as increasing maintainability, security, and flexibility.
However, a monolith presents a risk of the system being tightly coupled. The research done by the team (Assignment 2) documents tight coupling as one of the design risks; hence DIP was recommended as a design strategy.

Modular Monolith

Described as a software architecture that combines the benefits of modular design with the simplicity of a monolithic architecture, Modular monolith architecture involves dividing the system into a set of loosely coupled modules (Ozkaya, 2023).
The main benefit of such an architecture is the business logic is encapsulated in Modules, thus increasing reusability while data remains consistent and communication patterns simple (Ozkaya, 2023).

1. Team Capability

Considering Constraint CON-003, the team caps at exactly three students with individually evidenced contribution, and constraint CON-007 requires any architecture choice to be justified against "the actual capability of the team " (PED v1, Constraint Register). Comparing a modular monolith with an architecture like microservices, Milić & Makajić-Nikolić (2022) show that a monolith requires managing fewer moving parts per feature than microservices- for example, one Amazon Web Services (AWS) instance, one deployment pipeline, and constant security/deployment-profile counts regardless of how many features are added, versus these growing linearly with each additional microservice.

2. Deployment

Still on the example of an AWS instance, Milić & Makajić-Nikolić's metrics show the monolith needs 1 AWS instance, 1 pipeline, 3 deployment profiles, while microservices scale these k-for-k. Though this section is attributed to architecture and not necessarily deployment or hosting sites, using this example gives the team a real-world analysis and consideration of how a modular monolith performs.

Table 1. Software metrics' values—Monolithic software architecture.

Quality Attribute	Software Metric	Value
Coupling	Coupling Between Objects	16
Testability	Unit Tests	90
	Code Coverage (%)	94.2
	Covered Conditions	30
	Uncovered Conditions	0
Security	Security Hotspots	3
	Security Configurations	1
Complexity	Cyclomatic Complexity	75
	Cognitive Complexity	15
Deployability	Build Time (min)	02:07
	Deployment Pipelines	1
	Deployable Size (MB)	41.2
	AWS Instances	1
	Deployment Profiles	3
Availability	Healthcheck Endpoints	1

Source: Milić & Makajić-Nikolić (2022)

3. Complexity

Both reports from Huseynli (2026) and Milić & Makajić-Nikolić (2022) agree that a modular monolith trades distributed-systems complexity (network faults, eventual consistency, service-boundary erosion) for internal complexity (enforcing module encapsulation within one codebase). For CvicConnect's notification interaction, a network boundary, new failure modes, and new attack surface aren't justified, and the team lacks message-broker infrastructure, something modular monolith architecture solves.

Microservices Architecture

1. Team Capability

TaeiZadeh, Lotfi & Ramadhan's (2024) DevOps case study reinforces that successful Microservices architecture depends on changing the team culture, fostering collaboration between team members, requires the team to become cross-functional and multitask, and relies on DevOps team maturity as a selection criterion. This architecture is a poor fit because constraint CON-003 limits team member size to 3, and CON-007 requires justification against the actual capability of the team.

TaeiZadeh et al.'s own migration case still needed a large technical team plus a dedicated DevOps team with demonstrated maturity. CvicConnect has neither the headcount nor prior DevOps track record this literature treats as a precondition.

2. Deployment

Gudelli (2022/2026) explains that Continuous Improvement/Continuous Deployment (CI/CD) in microservices are inherently complex due to the need to coordinate deployments across multiple services, each with independent versioning, dependencies, and rollback requirements, requiring

contrast testing, feature flagging, canary/blue-green rollout, and automated rollback per service- a stark contrast to Milić & Makajić-Nikolić's constant "1 AWS instance/1 pipeline" monolith figures.
This architecture presents as a poor fit, as constraint CON-004 pushes free/low-cost hosting and flags deployment-limitation risk (PED v1, Constraint Interactions).

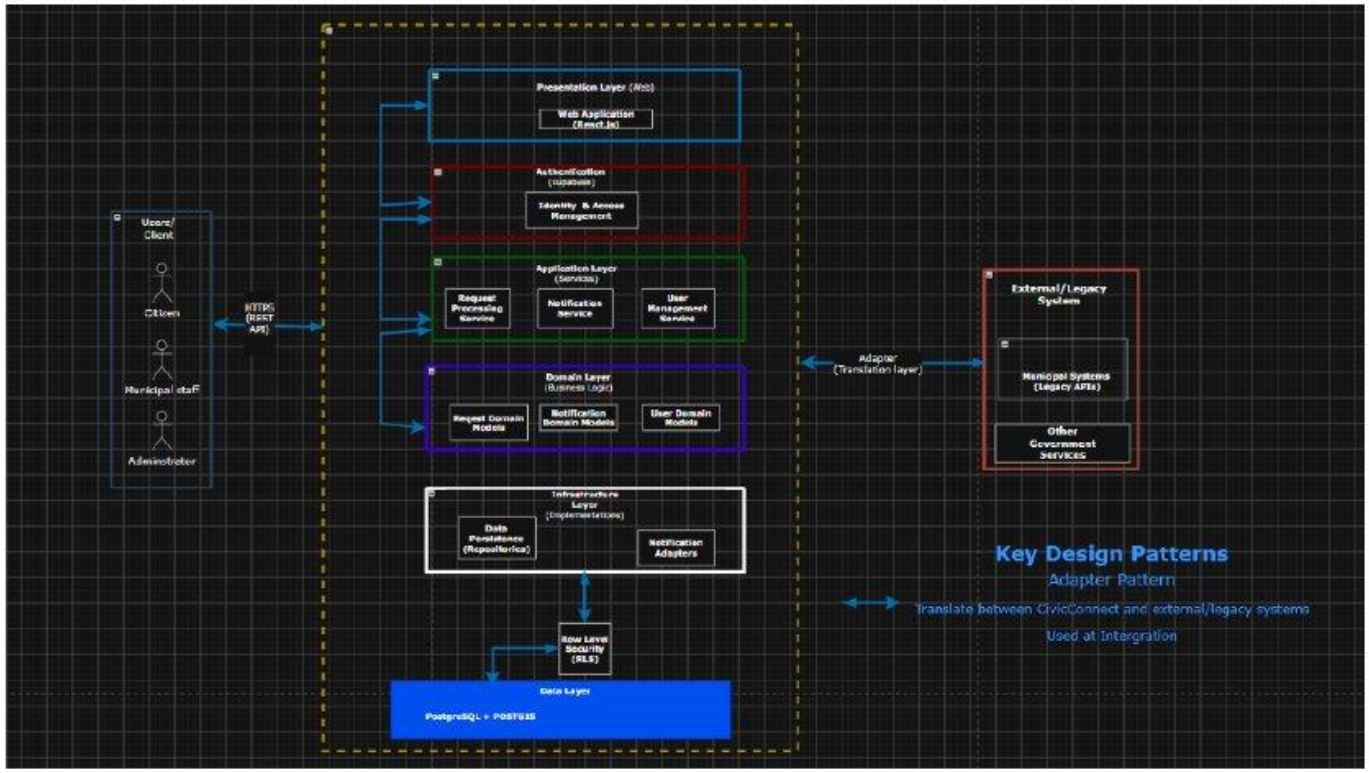
3. Complexity

TaeiZadeh et al. explain that there is a design-time complexity layer; even determining microservice boundaries is "tightly coupled with granularity level", has "ambiguous" metrics, and requires business-process ranking, service-call counts, and data-access-pattern analysis with expert interviews and a DevOps change-log review.
Introducing microservice-grade boundary-determination and orchestration complexity now would be exactly the "trend-driven adoption without sufficient contextual justification" both Husenyli and Gudelli warn against. CvicConnect has no equivalent evidenced load; PED v1 explicitly treats "the expected number of users, requests and stored data" as an open assumption (ASM-007) still to be established, not a known scale driver.
Recommendation
Therefore, **CivicConnect should adopt a Modular Monolith Architecture**. It provides:

- **Modularity**, supporting NFR-010 and allowing functionality to be separated into clear business-focused modules.
- **Manageable complexity**, appropriate for the three-member team specified by CON-003 and CON-007.
- **Simplified deployment**, because the modules remain part of one deployable application.
- **Lower infrastructure requirements**, supporting the free or low-cost preference established by CON-004.
- **Reduced coupling**, provided module boundaries and DIP are consistently enforced.
- **A better fit for the currently evidenced scale**, since ASM-007 does not yet establish demand that would justify microservices.

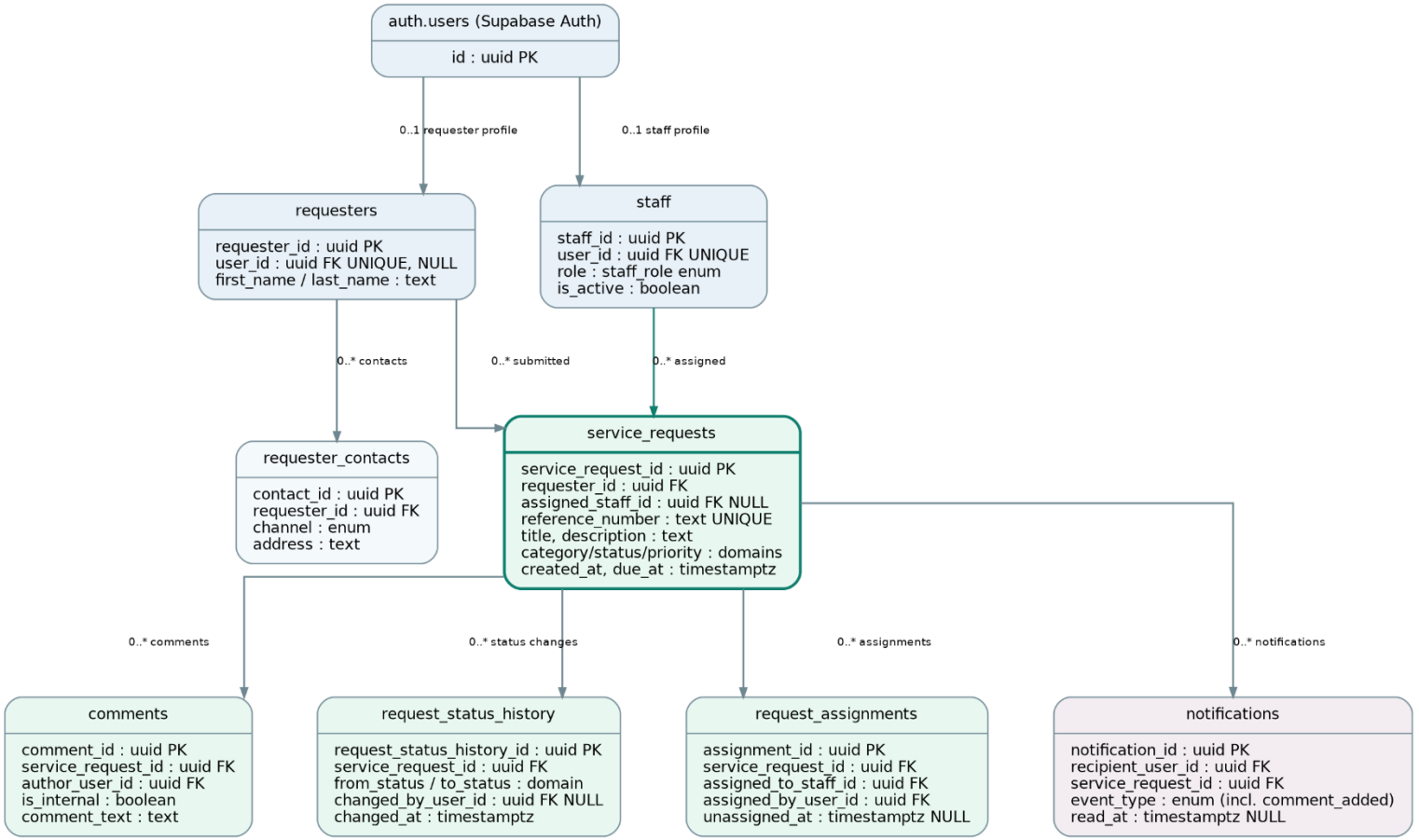
Overall, the **modular monolith offers the strongest alignment between CivicConnect's architectural quality requirements and its practical project constraints**. It retains the simplicity and cost advantages of monolithic deployment while addressing one of the principal weaknesses of a traditional monolith through explicit modular boundaries. The recommendation therefore represents a **context-driven architectural decision rather than adopting microservices primarily because of industry trends**.

Architecture Alternatives and Selection



Data and Persistence Baseline

ERD



The updated CivicConnect ERD includes the original identity and service-request entities, together with requester contacts, assignment history, notifications, attachments, channel messages and audit records.

The model reflects the current database structure, including the separation of authentication identities from application roles and the ability for both requesters and staff to comment on requests.

Structure

CivicConnect uses **PostgreSQL through Supabase** as its relational persistence technology.

The central identity table is `auth.users`, managed by Supabase Authentication. Application-specific information is stored in `requesters` and `staff`, with `requester_contacts` maintaining contact information separately. Staff roles include staff, manager and admin.

The main transactional table is `service_requests`, which stores each request, its requester, current assignee, category, priority, status and timestamps.

Supporting tables include `comments`, `request_status_history`, `request_assignments`, `status_transitions`, `notifications`, `attachments`, `channel_messages` and `audit_log`. These provide communication, historical accountability, notifications and support for different request-submission channels.

The database uses the following field types and storage sizes:

Data type	Size	Usage and reasoning
`UUID`	16 bytes	Primary and foreign keys; unique identification without sequential IDs
`TIMESTAMPTZ`	8 bytes	Created, updated, assigned and resolved timestamps; supports time-zone-aware records
`BOOLEAN`	1 byte	Internal comments, active staff and verification flags
`INTEGER`	4 bytes	Attachment sizes in bytes
`BIGINT`	8 bytes	Audit-log identity values
`ENUM`	4 bytes	Restricted role, channel and notification-event values
`TEXT`	Variable	Names, descriptions, comments and resolution summaries

PostgreSQL's `TEXT` type does not impose a fixed character limit. Suggested application limits are 100 characters for names, 150 for request titles, 5,000 for descriptions and 2,000 for comments. These are proposed validation limits rather than restrictions already enforced by the reviewed schema.

This design avoids unnecessary fixed-length fields while allowing the application to enforce reasonable input sizes.

Relationships

The relational model defines the following relationships:

- `auth.users` 1:0..1 `requesters` and `staff` — a user may have either or both application roles.
- `requesters` 1:* `service_requests` — a requester can submit multiple requests.
- `staff` 1:* `service_requests` — a staff member can be assigned multiple requests.
- `service_requests` 1:* `comments`, `request_status_history`, `request_assignments` and `notifications`.
- `requesters` 1:* `requester_contacts` — a requester can have multiple contact methods.
- `auth.users` 1:* `comments` — comments can be authored by requesters or staff.
- `service_requests` 1:* `attachments` — requests can have supporting files.
- `service_requests` 1:* `channel_messages` — messages can be linked to requests.

Relationships are enforced using foreign keys. The database also uses unique constraints, including a unique request reference number and a partial unique index preventing multiple current assignments for the same request.

Access Patterns

Requester:

Requesters insert service requests, retrieve their own requests, view status history, and read or submit public comments. Queries primarily filter using `requester_id`, with recent requests ordered by `created_at`.

Staff:

Staff retrieve assigned requests using `assigned_staff_id`, filter by status, priority or category, update permitted request statuses and add comments. Managers also require assignment-management and team-monitoring queries.

Management:

Managers and administrators require aggregated request data, staff workloads, assignment histories, overdue requests and reporting statistics.

The schema includes indexes on commonly queried fields, including `(requester_id, created_at)`, `assigned_staff_id`, `(status, priority)`, `category` and `due_at`. These support frequent filtering and ordering without requiring full-table scans for every operation. Indexes improve read performance but introduce additional storage and write overhead.

Integrity

PostgreSQL enforces referential integrity through primary keys, foreign keys, `NOT NULL`, unique constraints and `CHECK` constraints.

For example, a service request must reference an existing requester, while an assigned staff ID must reference a valid staff record. Comments must reference an existing request and an authenticated author. The database also rejects blank request titles, descriptions and comments.

Request assignment history is retained separately from the current assignment. A partial unique index ensures that a request cannot have more than one active assignment record.

Status changes and corresponding history records must remain consistent. These changes should execute within the same database transaction so that an unsuccessful history insertion cannot leave an unsupported status change.

Audit triggers and restrictions on modifying historical records further support accountability and data integrity.

Consistency

CivicConnect requires **strong consistency** for service-request status, assignments and historical records.

PostgreSQL provides ACID transactions, allowing related database operations to succeed or fail together. The current status is stored in `service_requests`, while `request_status_history` preserves earlier changes.

Similarly, `request_assignments` records assignment history while `service_requests.assigned_staff_id` identifies the current assignee.

Comment notifications are generated through an `AFTER INSERT` database trigger. This ensures that notifications are linked to successfully inserted comments and prevents notification creation from depending entirely on frontend behaviour.

The relational approach is suitable because the application requires accurate and immediately available operational data rather than eventual consistency between separate databases.

Sensitivity

CivicConnect processes personal information, including names, email addresses, contact details, request descriptions and communication records.

Authentication credentials are managed by Supabase Authentication rather than stored as plaintext passwords in application tables.

PostgreSQL Row Level Security (RLS) is used to restrict access according to authenticated identity and application role. Requesters should only access their own requests, while staff access is determined by assignment and management permissions.

Internal comments must not be visible to requesters. Sensitive contact details and audit records also require restricted access.

These controls support the confidentiality and security safeguards required by South Africa's **Protection of Personal Information Act (POPIA)**. Retention and deletion policies must additionally be confirmed before production deployment.

Expected Growth

CivicConnect initially targets a single community organisation, so expected data volumes are low to moderate.

For capacity planning, an illustrative workload of 100 new requests per day would produce approximately **36,500 requests annually**. At an average of five comments and three status-history records per request, this would generate approximately 182,500 comments and 109,500 status-history records annually.

These are planning estimates, not measured usage figures.

Actual storage will also depend on text lengths, indexes, audit records and attachments. Attachments are intended to use Supabase Storage, with PostgreSQL storing their metadata rather than the file contents.

PostgreSQL can support this anticipated workload without requiring database sharding.

Bottleneck / SPOF / Scalability / Availability / Backup

SPOF

The hosted Supabase PostgreSQL database is a critical persistence dependency. Database unavailability would prevent reliable request creation, retrieval and updates.

This risk is accepted for the initial deployment because a distributed database would introduce unnecessary operational complexity.

Backup and Recovery

Backup availability, retention and recovery options depend on the selected Supabase plan and configuration.

Before production deployment, the team must verify backup frequency, restoration procedures and acceptable recovery objectives. Database migration files must also be retained in version control to support environment reconstruction.

Scalability

The initial approach is to use a single hosted PostgreSQL database with appropriately indexed queries.

If usage increases, the team should monitor query execution times, database storage, connection limits and index effectiveness before increasing resources or introducing more complex architecture.

Availability

Application availability depends on Supabase's hosted infrastructure and the selected deployment configuration. Availability expectations should be checked against the hosting plan rather than assuming guaranteed uninterrupted service.

Persistence Model Decision

Relational — PostgreSQL through Supabase

A relational persistence model was selected because CivicConnect contains structured and strongly related entities requiring:

- Referential integrity between requesters, staff, requests and associated records.
- ACID transactions for status changes, assignments and notifications.
- Efficient filtering, joins and aggregation for operational dashboards.
- Role-based access through PostgreSQL Row Level Security.
- Historical records supporting accountability and auditing.
- Appropriate indexing and scalability for anticipated workloads.

PostgreSQL supports these requirements directly, while Supabase provides managed database hosting, authentication and storage integration.

A document-oriented NoSQL database was considered less appropriate because CivicConnect does not currently require schema-less or highly variable data structures. Its relationships, transactional requirements and reporting queries are better suited to a relational model.

The decision therefore balances **data integrity, security, performance, maintainability and implementation complexity** without introducing unnecessary distributed persistence infrastructure.

Technology-stack decision

M2-10 finalises the technology direction used for the current CivicConnect implementation. The choice was based on the approved requirements and ASRs, team capability, cost, maintainability, security and a realistic deployment path.

Selected M2 technology baseline

- UI/UX: Figma is used for manual design and prototyping only; AI-generated interface designs are not used.
- Frontend: React 19.2.8 with JavaScript, built with Vite 8.3.0.
- Backend and persistence: Supabase with PostgreSQL. Supabase Auth is selected for authentication; Node.js is used only as the tooling runtime, not as the application backend.
- Access control: role-based access with PostgreSQL RLS is the selected direction, but RLS is intentionally not enabled yet. It will be configured after authentication and live database integration are stable.
- Testing and CI: Vitest 5.0.2, npm, Git/GitHub and GitHub Actions are used for repeatable verification.
- Deployment direction: Vercel for the React/Vite frontend with Supabase remaining the hosted backend/database platform.

Why this stack fits CivicConnect

React supports the interactive forms and later dashboards required by CivicConnect, while PostgreSQL fits the project's structured relational data. Supabase reduces the amount of custom backend infrastructure a three-person team must build and maintain. The selected tools also support low-cost development, automated checks and a straightforward path to deployment.

Current M2 status

- Implemented: React/Vite client, request-submission logic, Supabase client configuration, PostgreSQL migration, unit tests and GitHub Actions CI.
- Still planned: live Supabase authentication, live database writes, final role/RLS policies and later component tests.
- ADR-001 records the technology decision and its main trade-offs.

Initial design decisions — informed by A2 research

Design Problem 1 – Excessive Dependencies between CivicConnect Components

Based on the research conducted for Assignment 2, the most prominent problem requiring high-priority attention was components or modules being highly reliant on one another, resulting in tight coupling. This presents a design problem because one of the requirements identified in the PED is that the solution

should use a modular structure so that authorised team members can test one area without affecting other parts of the application. **Failing to address this problem could make the project unmaintainable** for the reasons discussed above. **Candidate 1- Dependency Inversion Principle (DIP).**
The Dependency Inversion Principle states that high-level modules should depend on abstractions rather than low-level modules, with both depending on abstractions (Ramachandrappa, 2024). For CivicConnect, this means that services such as Request Processing, Notifications, and Data Persistence should communicate through interfaces rather than directly depending on concrete implementations.

Structure and Responsibilities

- High-level modules depend on abstractions.
- Low-level modules implement those abstractions.
- Components communicate through interfaces.

Benefits

According to Yadati (2023):

- Reduces complexity.
- Reduces coupling.
- Makes dependency-related problems easier to identify.
- Promotes modular system design.

Limitations

According to Yadati (2023):

- Requires more architectural planning.
- Can introduce additional abstraction layers.

Evaluation (From Assignment 2)

DIP provides low coupling because components rely on abstractions rather than direct implementations (Yadati, 2023). Cohesion is high because each service focuses on its own business responsibility (Ramachandrappa, 2024). Maintainability is also high because implementation changes do not necessarily affect dependent modules. With this approach, testability is rated as high.
The main trade-off is that DIP introduces additional interfaces and abstractions, which can increase complexity, particularly when abstractions are introduced without a genuine need.

Candidate 2- Adapter Pattern

The core idea is that this is a mechanism that resolves interface incompatibility between client and third-party/legacy system, acting as a “translation layer” between interfaces without altering either system (Ekhlakov, 2025).

Structure and Responsibilities

According to Ekhlakov (2025), the adapter converts one interface into another without altering either system. For CivicConnect, this would be useful if:

- Citizen Request Services must integrate with legacy municipal systems.
- Notification systems expose different APIs.
- Third-party government or payment services provide incompatible interfaces.

Benefits

According to Ekhlakov (2025):

- Minimal changes required to existing code.
- Improved readability.
- Preserves compatibility with external systems.

Limitations

According to Ekhlakov (2025):

- Introduces an additional translation layer.
- Additional abstraction can slightly increase complexity.

Criterion	**Assessment**
Coupling	Low
Cohesion	High
Maintainability	Very Good
Extensibility	Good
Testability	Good
Benefits	Enables integration of incompatible systems with minimal code changes.
Limitations	Additional adapter later to maintain.

Architecture context

M2-03 selects a Modular Monolith for CivicConnect's overall architecture, on the basis that it satisfied 7 of 8 ASRs (Architecturally Significant Requirements) and fit CON-003, CON-004, and CON-007 better than microservices at CivicConnect's current evidence scale.

- In a modular monolith, all internal components run in the same process. There is no network between them, so any coupling-reduction pattern applied internally is solving an in-process dependency problem, not a distributed-systems one.
- Reasons for selecting the adapter pattern, as justified in assignment 2, were never about internal components, but rather about the boundary with external, legacy municipal systems, which sit outside the modular monolith's boundary entirely. The two selected architectures are not competing for the same function or purpose, but they solve two different sub-problems that both happen to fall under "Design Problem 1".

Final recommendation

Assignment 2's final recommendation is to adopt both DIP and the Adapter Pattern for Design problem 1, instead of either one. Below are the reasons why (based on the research done in Assignment 2):

- DIP for internal CivicConnect component relationships (Request Processing, Notifications, Persistence), to reduce unnecessary internal dependencies and support NFR-010 (maintainability: modular structure, change one area without unnecessary changes to unrelated areas).
- Adapter Pattern specifically for the boundary with external/legacy municipal systems where interface incompatibility- not just internal coupling- is the actual problem.

Applying DIP to every component could introduce unnecessary interfaces; creating an Adapter where no interface incompatibility exists provides little value. The Adapter Pattern is a recommended approach mainly in areas where CivicConnect must communicate with existing municipal legacy systems or external services.

Design Problem 2: Components Having Too Many or Unrelated Responsibilities

As CivicConnect's capabilities expand, individual components may begin to take on numerous, unrelated responsibilities instead of maintaining a single, focused role. Concretely, as request data progresses through the system—from creation, validation, categorisation, and persistence to status changes and notifications—a single class or service may gradually take on responsibilities involving business logic, validation, persistence, and notification simultaneously. This often occurs because it is easier to add "just one more thing" to an existing component rather than develop a new one.
This is a genuine design problem for CivicConnect because:

- **FR-005** (in-app notifications on status change) and **FR-010** (staff updating request status through approved workflow stages) both interact with the same underlying request record. This could lead to a tendency for one component to manage both responsibilities instead of maintaining a separation between status-update logic and notification-triggering logic.
- **NFR-010** (Maintainability) clearly states that authorised team members should be able to test and modify one area without affecting other parts of the system. A component that manages multiple unrelated concerns contradicts this requirement, as altering notification behaviour could inadvertently affect the status-update logic contained within the same class.
- **CON-003** (3-person team) indicates that there is no specific "notifications team" or "validation team" to informally maintain separation through team structure. Instead, the design of the codebase itself must ensure this separation.

Candidate 1 – Strategy Pattern
The **Strategy Pattern** encapsulates a family of related algorithms or behaviours into separate classes, with each class handling a specific responsibility. The appropriate strategy can be selected dynamically at runtime rather than embedding all behaviours within a single component.
Benefits
The benefits of using this pattern include reduced complexity and improved modularisation, resulting in a more maintainable and adaptable system (Syed-Mohamad et al., 2026). Afteni (2025) states that the Strategy Pattern improves readability and maintainability, further noting that it makes classes easier to refactor, test, and extend, thereby reducing the likelihood of introducing errors.
Limitations
However, this pattern does have limitations. One limitation is the close internal dependency between the client and the strategies (IONOS Editorial Team, 2023). IONOS Editorial Team (2023) further notes that the provided interface is not always used optimally in the Strategy Pattern.
Evaluation

Criterion	Assessment
Structure/Responsibilities	Separate strategy classes for each business behaviour.
Coupling	Very low.
Cohesion	Very high.

Maintainability	Excellent.
Extensibility	Excellent.
Testability	Excellent.
Benefits	Flexible behaviour changes without modifying existing code.
Limitations	Increased number of classes.

Candidate 2 – Façade Pattern

The **Façade Pattern** provides a cohesive, high-level interface for a group of related classes, concealing their internal complexities behind a more straightforward access point (Goblirsch-Urban, 2024). For CivicConnect, rather than exposing multiple internal responsibilities directly to clients, a Façade could provide a single, simplified interface for interacting with related services.

Benefits

The benefits of this solution include simplifying interactions between components, preventing clients from needing to understand internal implementations, and supporting maintainability when internal services change.

Trade-offs

The trade-off is that the Façade itself may become too large as multiple responsibilities are routed through it, while also becoming a bottleneck if poorly designed. Furthermore, it does not eliminate responsibilities; rather, it organises access to them.

Evaluation

Criterion	Assessment
Structure/Responsibilities	Single entry point coordinating multiple focused services.
Coupling	Low client-to-subsystem coupling.
Cohesion	Moderate to high when the Façade coordinates a coherent subsystem.
Maintainability	High when internal services can change without affecting clients.
Extensibility	High when additional internal services can be incorporated behind the interface.
Testability	Good, although the Façade may require tests covering its coordination logic.
Benefits	Simplified interaction and reduced client-facing complexity.
Limitations	Risk of becoming an oversized Façade containing excessive responsibilities.

Architecture Context

The **Modular Monolith architecture** maintains distinct internal modules while functioning as a single deployable system. This structure is particularly relevant to **Maintainability (NFR-010)**, as it allows authorised team members to modify specific areas without unnecessarily affecting unrelated sections.

This selection does not mean that a **Microservices architecture** is unsuitable in all circumstances. However, if CivicConnect's integration requirements or operational needs change, the architecture can be revised through the project's change-control process. The current baseline supports the Modular Monolith as a suitable architecture, providing the required architectural boundaries without introducing unnecessary complexity.

Final Recommendation

Assignment 2 identifies the **Strategy Pattern** as the recommended approach where CivicConnect must support multiple alternative behaviours or decision rules that can change independently, rather than embedding numerous conditional branches within a single component. With this approach, new behaviours can be introduced without substantially modifying the component that uses the strategy.

For CivicConnect, this could allow different request-related behaviours to be encapsulated within separate strategy classes, ensuring that each strategy has a focused responsibility. This supports **NFR-010 (Maintainability)** by reducing the likelihood that changes to one behaviour will unnecessarily affect unrelated functionality.

The **Façade Pattern** may still be appropriate where CivicConnect requires a simplified interface for coordinating multiple related services. However, it does not directly address the underlying problem of separating alternative behaviours into focused responsibilities. Therefore, its use should be limited to situations where simplifying access to a group of services provides a clear architectural benefit.

Initial API / integration decisions where applicable

(ADR-008) Decision 1:

Interaction:

Client (Requester/Staff UI) communication with Backend (Request Handler Service).

Responsibilities and Information exchanged:

The requester/staff is the client who is the caller in this interaction, and the backend is the provider the callee. The client submits new requests, retrieves current status and request history, and staff submit status updates and filtered queries. Which are FR-001, FR-003 and FR-010 respectively.

The information exchanged would be the details of the requests when submitted. The status, category and priority of requests when retrieved. The authentication credentials and session token details on every call.

Mechanism:

HTTP/REST API

The client in the app/browser is its own boundary while the backend is another they do not operate on the same runtime. The information will need to be exchanged through the boundaries. Therefore, a network-based mechanism is justified here. This decision is made based off the research done Assignment 2 Task 3 where the REST principles were evaluated. So based off the research on HTTP/REST API it is the best fit for this interaction.

Validation:

The server side will enforce validation checks on top of the client-side checks. The request fields will be validated against the required fields such as not null, and matches the valid categories as per the DEC-005 of having DB-level CHECK/ENUM constraint, not a lookup table, as well as other checks.

Error/Failure Behaviour:

- 400 for invalid input.
- 401/403 for authentication/authorisation failures which links to NFR-001
- 404 for a request not found.
- 500 for server-side failure.
- Client must handle timeouts/dropped connections distinctly from a definitive rejection.

Security Boundary:

Requests must have valid session credentials. The server-side authorisation checks will implement Role Based Access Control (RBAC) to ensure that the caller has access to and can interact with requested resource.

Change/Version Implications:

The caller and provider can be updated independently. A change that breaks away from the request/response shape requires coordinated deployment or explicit API versioning once multiple clients exist.

Evidence:

A2 Task 3's Research on the REST mechanism and the evaluation of its REST principles.

(ADR-009) Decision 2:

Interaction:

Status Update (Request Handler) produces and sends an in-app Notification.

Responsibilities and information exchanged:

The request handler service is the producer that produces a notification that is consumed by the notification component.

Information exchanged is when a status is changed the producer will publish the requestID, the new status and the requester involved. The consumer reacts by sending the notification of the update to the requester.

Mechanism:

In-process asynchronous event (observer pattern)

Both components run in the same deployable unit therefore it is an internal interaction. HTTP would be an unjustified network boundary which was discussed and reasoned in Assignment 2 Task 3. Due to the research and findings along with the properties of the interaction it reasoned that In-process asynchronous event observer pattern is the most suitable mechanism.

Validation:

The event payload must reference a valid requestID with a status value that is valid and matches a status in the controlled set such as the ones mentioned in FR-014. And will similarly be checked and validated like the categories with the DB-level CHECK/ENUM constraint that DEC-005 spoke of. Which must be enforced by the producer be it publishes.

Error/failure behaviour:

This runs I process so failure wont really come from a network timeout as it will fail as part of the same call stack. The producer's status update transaction, the status and the required comment of change decision that was made with regards to auditing in DEC-007 must be completed before the event fires so that if the update fails a false notification will not be triggered.

Security Boundary:

There are no new endpoints, so the applications access control enforces the status update action itself.

Change/Version implications:

Adding a new consumer such as the FS-002 deferred scope of the SMS, WhatsApp and email integration (that we are looking into possible simulation options for now) will require zero change to the producer which is the main advantage that this mechanism was chosen for so we also have the option.

Evidence:

Assignment 2 Task 3 the mechanism comparisons that led to this mechanism being chosen for this interaction due to its properties and opportunities for growth. The DEC-006 also relates to this as there is no Notification persistence as the notification is once off at update and carries no data we might need to use.

Unestablished Interactions:

FEC-004 will be updated to reflect considerations of the interactions of the external WhatsApp, SMS, and email implementation and how we could possibly integrate them similarly to the Decision 2 described here as a consumer.

Deployment compatibility

M2-11 checks that the selected architecture, data and technology choices can be deployed without claiming that production deployment is already complete.

Current deployment direction

- Frontend: build the React/Vite client and host it on Vercel. Netlify remains a viable alternative if later project evidence justifies a change.
- Backend/data: Supabase remains the managed backend platform, providing PostgreSQL and the selected authentication direction.

- Communication: the deployed browser client will communicate with Supabase over HTTPS. No separate Node.js/Express application server is required for the current scope.
- Environments: local development is active now; review happens through Pull Requests and CI; production deployment remains deferred.

Configuration and security

- The client uses VITE_SUPABASE_URL and VITE_SUPABASE_ANON_KEY through environment configuration. An .env.example documents the required names while real .env files remain excluded from source control.
- Privileged Supabase secrets must never be placed in the React frontend or committed to Git.
- RLS is not enabled at this stage. Authentication, role mapping and RLS policies must be completed and tested before production use.

Deployment risks and deferred decisions

The main deployment risks are incorrect environment configuration (R-013), exposing a privileged key (R-014), dependency on hosted-service availability/limits (R-015), and behaviour that differs after deployment, especially authentication or access control (R-016). Production domain/DNS, paid plans, final scaling, backup/recovery settings, monitoring and automatic Vercel deployment are intentionally deferred until the team has enough usage and deployment evidence.

SCM and CI adoption (M2-12)

M2-12 applies the collaboration and CI practices that are useful at the current stage. The aim is controlled integration and repeatable verification, not a mature CI/CD pipeline.

Repository workflow

- Working branches are merged into staging through Pull Requests; validated staging changes are later promoted to main.
- Substantive Pull Requests require two independent reviewer approvals. Documentation and code changes both go through review.
- Issues, branches, commits and Pull Requests are used together to keep the work traceable, while force pushes and unsafe direct changes to controlled branches are restricted.

Initial GitHub Actions CI

- Workflow: .github/workflows/client-ci.yml.
- Triggers: Pull Requests into staging/main and pushes to those controlled branches.
- Checks: npm ci, npm test, npm run lint and npm run build using Node.js 24 from src/client.
- Verification: three Vitest request-service tests pass, lint passes and the Vite production build succeeds. The Client CI / verify-client check passed on Pull Request #77 before the workflow was merged into staging.
- Secrets are not stored in the workflow. Real environment files remain excluded from source control.

Deferred after M2

Dependency/security auditing in CI, full continuous deployment and automatic Vercel deployment are left for a later milestone. Human review remains required even when CI is green.

Meaningful development and application evidence (M2-13)

M2-13 moves CivicConnect from planning into controlled construction. The repository now contains project-specific application code, persistence artefacts and automated verification rather than only framework scaffolding.

Implemented in M2

- A React/Vite client exists under src/client with an initial service-request submission form.
- requestService.js validates category, priority, title and description, creates a unique reference number and sets the initial status to Submitted.
- supabaseRequestRepository.js provides the persistence boundary and is aligned to the current relational schema.
- The PostgreSQL/Supabase migration defines requesters, staff, service_requests, request_status_history and comments. Category and status are stored directly on service_requests, and priority is included.
- The Supabase client and .env.example configuration are present without committing real credentials.
- The schema supports requester/staff ownership, assignment, comments and status-history auditing even though those full user flows are not implemented yet.

Verification evidence

- requestService.test.js contains three automated tests for valid request creation, required fields and trimmed input.
- npm test passes all three tests; npm run lint passes; npm run build produces the Vite production build.
- M2-12 repeats the same checks through GitHub Actions, confirming that the verification is repeatable outside the developer machine.

Current limitations / controlled future work

- Supabase Auth is selected but is not connected to the current request flow.
- Live request persistence is not yet wired to the form; the repository boundary exists for the later integration.
- RLS policies are deliberately deferred until authentication, roles and live database integration are ready.
- Staff assignment, status-update/comment screens, requester history, management dashboards and production deployment remain later development work.

Current end-to-end trace

FR-001 Submit Service Request → data-integrity/reliability/maintainability drivers → request-submission responsibility → service_requests persistence → separated UI/service/repository design → React/Vite + Supabase/PostgreSQL (ADR-001) → RequestForm.jsx, requestService.js, supabaseRequestRepository.js and 001_initial_request_schema.sql → request-service tests plus CI verification. The requirement is therefore In Development rather than fully Implemented because live authentication and database persistence are still pending.

Section C:

M3 Presentation Link:

https://belgiumcampusacza-my.sharepoint.com/:v:/g/personal/603090_student_belgiumcampus_ac_za/IQCfHcaOeXYhS5SCstyQu3niAdUEdvdlGy-FgAVwaw_O5oo?e=j43lYu

M3 Controlled Construction, Verification and Release Evidence

CivicConnect's M3 baseline continues the earlier requirements, architecture and persistence decisions. This section records the integrated behaviour demonstrated while retaining unverified claims and outstanding risks as explicit follow-up work.

C.1 Implementation status and quality evidence

Area	Observed state / evidence	Verification gap
Functional scope	Requester submission, staff processing, manager assignment/oversight, comments and notification workflows demonstrated.	Complete permission-negative testing; fresh deployment verification.
Automated test code	83 cases previously identified in the submitted ZIP (80 normal, 3 opt-in database tests).	Attach actual run output, versions and RC commit for the 80 default cases.
Real database integration	Screenshot reports all 3 opt-in Supabase integration tests passed.	Attach dated test-run photograph/report and associate it with the RC version.
CI, security and performance	ESLint, Vitest and build scripts exist; no final passing CI/performance evidence supplied.	Collect CI job, secret/dependency check and small load-test results.
Staging/deployment	Development Supabase environment supports live UI demonstration.	Reconcile SQL functions/triggers with controlled migrations; verify fresh staging installation.

C.2 Selected meaningful unit/component and integration tests

ID	Test and engineering significance	Evidence classification
UT-01	Dual requester/staff identity recognition prevents one account being incorrectly treated as a single-role account.	Existing Vitest test; separate execution summary needed.
UT-02	Legacy supervisor role rejection checks that the retired role is not accidentally treated as current.	Existing Vitest test; schema seed must still be reconciled.
UT-03	Request creation produces a reference number and Submitted state, checking essential workflow defaults.	Existing Vitest test.

ID	Test and engineering significance	Evidence classification
UT-04	Failed status update must not emit success, preventing false acknowledgements on rejected operations.	Existing mocked test; does not establish transaction rollback.
UT-05	Duplicate submission retry does not create another logical request, protecting data quality.	Existing mocked test; database test adds stronger evidence.
INT-01	Real request persists and duplicate retry does not produce a second record.	Teammate-reported PASS
INT-02	Database rejects unsupported request category independently of frontend checks.	Teammate-reported
INT-03	Unauthenticated database INSERT is rejected by privileges/RLS.	Teammate-reported

The real database integration tests demonstrate stronger persistence and access-control assurance than mock-only tests, but do not by themselves prove manager or administrator least-privilege rules. The teammate's screenshot should be added to the controlled evidence folder before sign-off.

C.3 Manual black-box tests: five executed and passed

Case	Technique and test	Actual outcome / evidence
BB-01	Equivalence Partitioning / Boundary Value Analysis: invalid registration email, seven-character password, valid signup.	PASS — validation errors and signup confirmation shown in supplied screenshots.
BB-02	Boundary Value Analysis: mandatory request fields, title length at the accepted boundary.	PASS — required-field validation and valid request shown; screenshot alone does not establish database maximum-length constraint.
BB-03	Equivalence Partitioning: requester B attempts direct URL access to requester A's record.	PASS — "not found or not accessible" screen shown.
BB-04	Decision Table: staff public comment vs internal note; requester reply and correct notification recipient.	PASS — staff/requester views and "New Comment" notification shown.
BB-05	State Transition Testing: allowed status updates and chronological request history.	PASS — Accepted → In Progress → Resolved and history shown; direct-RPC invalid transition not yet tested.

Black-box techniques were selected for relevance rather than volume. EP and BVA address form-input partitions/boundaries, decision-table logic addresses comment visibility, and state-transition testing addresses the request lifecycle. Client-side blocking is not equivalent to server-side enforcement.

C.4 End-to-end, security and operational checks

Case	Recorded result	Engineering interpretation / next action
E2E-01	PASS	Core requester → manager → staff request lifecycle, comments, notifications, status and assignment history demonstrated in the UI.
E2E-02	PARTIAL / NOT PASSED	Admin staff record creation now succeeds; invitation email received and link reaches Set Password. Completion of password setup and first staff login not confirmed. Consumed link cannot be resent from existing Add Staff workflow.
SEC-01	BLOCKED	Second manager session not tested; can use normal browser + Incognito on same localhost installation.
SEC-02	NOT VERIFIED	Manager status updates on another assignee require negative authenticated RPC testing.
SEC-03	UI ONLY	Admin cannot assign self through UI; database denial still requires direct authenticated test.
SEC-04	PASS (UI)	Manager cannot claim an unassigned request through the manager processing interface.
INT-04	REPORTED PASS (3/3)	Teammate confirmed the dedicated Supabase integration tests passed; attach screenshot and configuration metadata.
OPS-01	BLOCKED	Fresh database recreation is hindered by SQL functions deployed manually but not yet reconciled with committed migrations.

C.5 Evidence photographs (selected excerpts)

The images below are excerpts from the evidence document and support the specific UI claims in the tables. They are not substitutes for CI output, backend permission tests or integration-test execution logs.

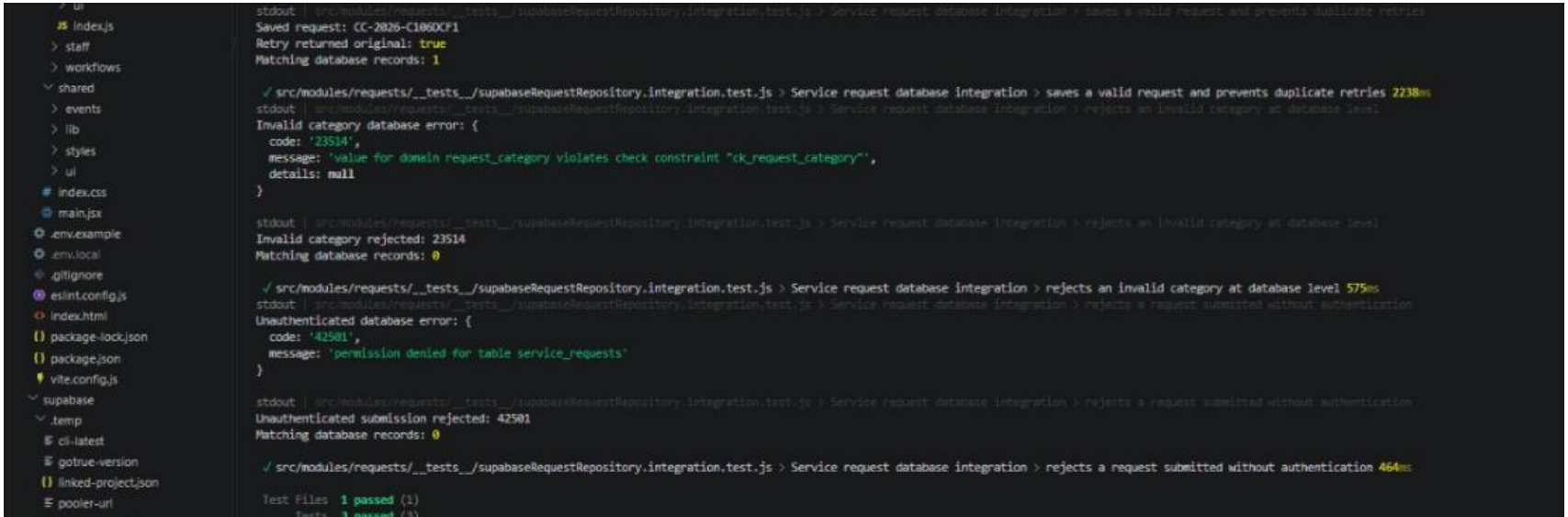


Figure C-1. BB-01 — invalid email validation.

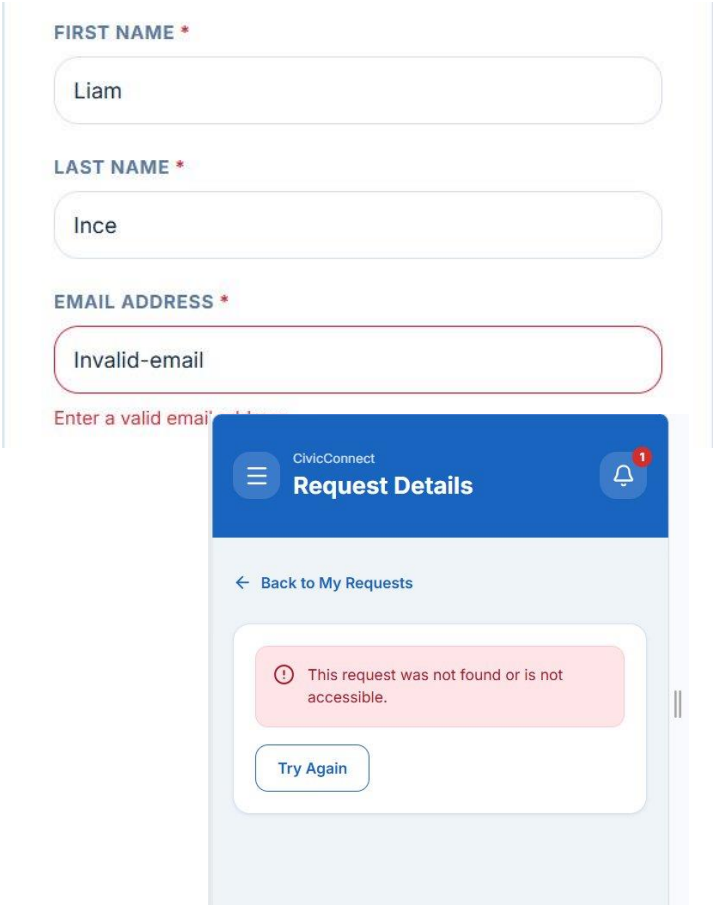


Figure C-3. BB-03 — other requester denied access.

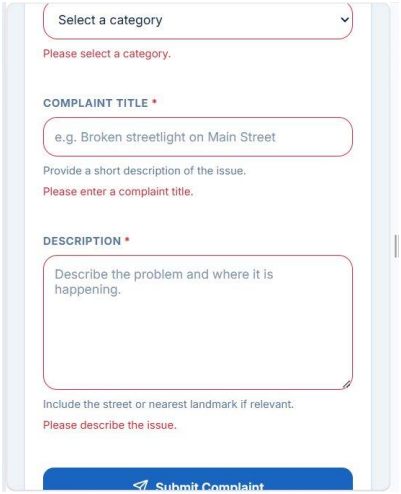


Figure C-2. BB-02 — required request field validation.

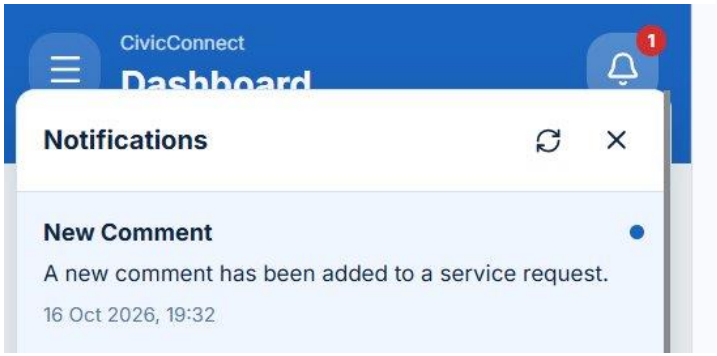


Figure C-4. BB-04 — new-comment notification.

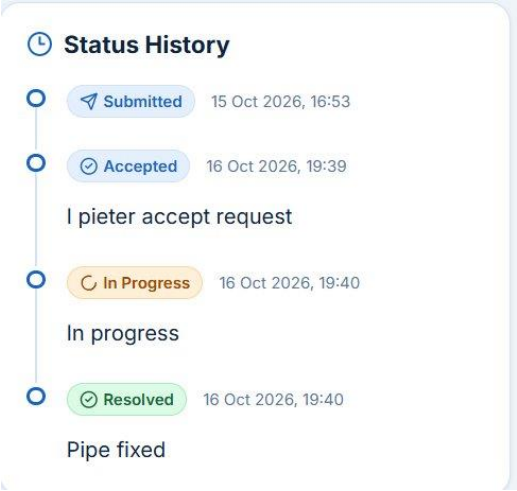


Figure C-5. BB-05 — ordered request-status history.

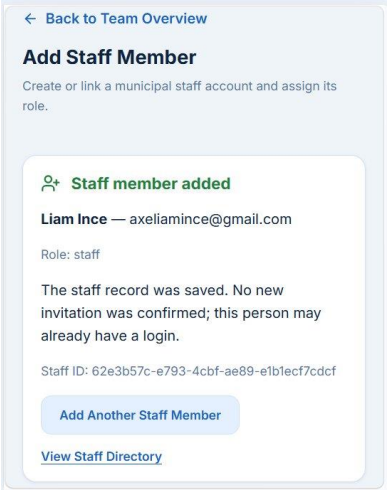


Figure C-6. E2E-02 — existing account added as staff.

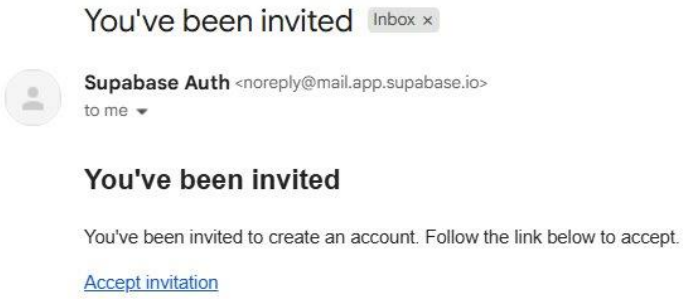


Figure C-7. E2E-02 — new invitation email received.

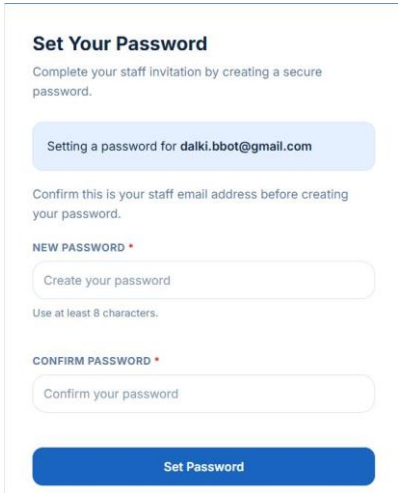


Figure C-8. E2E-02 — reached Set Your Password page.

Evidence-source note: images contain test account information as originally supplied. Redact personal identifiers before distributing the PED beyond the authorised assessment team. The originally captured permission-error screen and earlier localhost:3000 redirect are retained in the complete evidence document.

C.6 Release-readiness judgement

Provisional decision: CONDITIONAL GO for a supervised M3 staging demonstration, subject to explicit completion of the remaining security and evidence gates; NO-GO for public production deployment in the currently verified state.

Basis: the five selected black-box tests and the core E2E journey passed in the submitted manual evidence. Three real database integration tests were also reported as passed by a teammate. However, E2E-02 has not completed first staff-login validation, management permissions are not proven at the database boundary, and a fresh Supabase deployment cannot yet be reproduced solely from controlled migrations.

Gate	Release condition / evidence needed	Status
G-01	Finish invited staff password setup, login and staff-role restrictions; provide a recovery/resend flow.	OPEN
G-02	Negatively test manager/admin assignment and status RPC permissions with authenticated user sessions.	OPEN
G-03	Reconcile and test all schema migrations/functions against a fresh disposable database.	OPEN
G-04	Attach real integration test screenshot and default automated test + CI results for exact RC commit.	OPEN
G-05	Run one small justified performance test and security/dependency/secrets check.	OPEN

A conditional staging demonstration is a limited academic assessment decision, not approval to expose municipal users or personal data to a public production service. Known critical authorisation defects must be corrected rather than accepted as residual risk.

C.7 Residual risk statement

Risk	Why it remains / current mitigation	Owner / next action
Real production workload unknown	Student-scale demonstrations and tests do not establish municipal-scale throughput. Existing indexes support ordinary lookups.	QA/Backend: perform representative load test and review query plans before live rollout.
Supabase service or email availability	Authentication, database and invitations depend on managed external services. Errors are surfaced to the UI.	Deployment: agree response, backup and retry procedures.

Risk	Why it remains / current mitigation	Owner / next action
Invitation consumption without recovery	One-time link behaviour is expected for security; lack of staff re-invitation path can strand an already provisioned account.	Backend/Frontend: implement controlled resend/recovery (TD-09).
Manual SQL deployment drift	Database-only changes may diverge from Git migrations; development may work while clean deployment fails.	Backend: migration parity audit and clean environment verification (TD-02).

C.8 RTM and controlled-evidence pointers

Demonstrated in controlled artifact as well on github.

Requirement / concern	Trace to implementation and test	Evidence location
Submission and confirmation	Request UI → request service/repository → service_requests → BB-02, UT-03, INT-01, E2E-01	TestEvidenceCivicconnect(1).docx; Vitest report.
Ownership/access security	Auth/RLS → BB-03, INT-03; management negative cases SEC-01–03 pending	Evidence screenshots + pending SQL tests.
Comments and notifications	comments / notification trigger → BB-04, E2E-01	Evidence screenshots and SQL migration files.
Status and assignment audit	status history + request_assignments → BB-05, E2E-01	Status/assignment history evidence.
Admin staff provisioning	Add Staff Edge Function / Auth invite → E2E-02 pending	Success/invite/set-password screenshots; TD-09 and defect.
Deployment repeatability	Version-controlled migrations, Git history, CI → OPS-01 pending	Repository migrations, CI/staging evidence to be added.

PED v3 is the evidence index, not a substitute for authentic repository history. Include the RC commit/tag, updated RTM, issue and PR IDs, named test execution evidence, release review sign-off and staging demonstration instructions before submission.

Section D:

Controlled Artefacts:

Stakeholder Register:

RTM:

Controlled evolving artefact in Github repo and will be shown upon presentation
CivicConnect/docs/requirements/CivicConnect RTM M2.xlsx at staging · RyanTPH/CivicConnect

Risk Register:

Controlled evolving artefact in Github repo and will be shown upon presentation
CivicConnect/docs/risk/CivicConnect Risk Register.xlsx at staging · RyanTPH/CivicConnect

Forward Engineering Decisions

Controlled evolving artefact in Github repo and will be shown upon presentation
CivicConnect/docs/decisions/CivicConnect Forward Engineering Decisions.xlsx at staging · RyanTPH/CivicConnect

Decision Log/ADR:

Controlled evolving artefact in Github repo and will be shown upon presentation
CivicConnect/docs/decisions/CivicConnect Engineering Decision Log.xlsx at staging · RyanTPH/CivicConnect

Technical Debt Register:

Will be recorded as an artifact as well on github.

ID	Category / concrete debt	Consequence, priority and response
TD-01	Security: management RPC authorisation mismatch	P0: manager ownership/admin processing rules need server-side enforcement and negative RPC tests. Backend owner; release gate.
TD-02	Infrastructure: manually deployed SQL not fully migrated	P0: clean environment cannot be reconstructed reliably. Backend owner; reconcile functions, grants, triggers and migration order.
TD-03	Test debt: E2E and RPC regression coverage gaps	P0 for release: record negative role tests, fresh deployment checks and repeatable E2E evidence. QA/Backend.
TD-04	CI evidence not available for submitted RC	P0 for evidence: confirm lint/test/build workflow and link passing job for exact commit. CI owner.
TD-05	Legacy supervisor role still appears in foundation SQL	P1: role-model drift may cause seed/permission inconsistencies. Backend: controlled enum/seed migration.
TD-06	Text input constraints not consistently DB-enforced	P2: API clients can bypass UI length checks. Backend/Frontend: approve targeted constraints, test migrations.
TD-07	Recovery objectives and backup plan unverified	P1 before production: document Supabase plan and restoration test. Deployment owner.
TD-08	Dual-role account portal choice not fully tested	P1: existing requester becoming staff may still be redirected to requester portal. Frontend: portal switching and login tests.
TD-09	Staff invite cannot be recovered/resubmitted from Add Staff after provision	P1 (P0 if onboarding is mandatory for RC): invitation link is single-use, while Auth invitation and staff row creation are separate steps; retry returns “already staff”, leaving no resend path. Implement idempotent invitation-state workflow, admin resend/recovery action and positive/negative tests; no false claim of one DB transaction across Supabase Auth and PostgreSQL. Backend/Frontend.

Change Request Register:

Will be recorded as an artifact on github.

Change ID	Design change and rationale	Impact / verification status
CR-PS-01	Separate requester/staff profiles, linked auth identities and requester contacts; supports multirole and non-login requesters.	Identity schema, signup provisioning and RLS; code exists, fresh migration reconciliation pending.

Change ID	Design change and rationale	Impact / verification status
CR-PS-02	Expand service request lifecycle, source/due/resolution fields and from/to status history.	Request status RPC, dashboards/reports and timeline; BB-05/E2E-01 passed UI evidence.
CR-PS-03	Add current-assignee plus request_assignments history for accountable reassignment.	Management pages/RPC, partial uniqueness; E2E-01 demonstrated, negative role tests outstanding.
CR-PS-04	Allow public requester/staff comments via author_user_id and private staff notes via is_internal.	RLS and comment UI; BB-04 and E2E-01 passed.
CR-PS-05	Add persistent notifications and comment_added trigger for public-comment communication.	Notification enum/function/bell; manual comment event demonstrated.
CR-PS-06	Introduce attachments, channel_messages, audit_log and supporting constraints.	Additional migrations/RLS/retention considerations; production channel scope deferred.
CR-PS-07	Complete admin provisioning: server-side permission RPCs, account linking and invite-based password setup.	Admin Edge Function/identity tables; staff row + invite + password page demonstrated, full activation and resend recovery outstanding (TD-09).

AI Usage Register:

CivicConnect/docs/ai/AI Usage Register.docx at staging · RyanTPH/CivicConnect

Github Repo Link:

https://github.com/RyanTPH/CivicConnect

References

Belgium Campus ITversity, 2026 a. *CivicConnect Master Project Brief: Community Service Request Management Platform,* Pretoria: Belgium Campus ITversity.

Belgium Campus ITversity, 2026 b. *CivicConnect Project Milestone 1: Engineering Foundation and Requirements Baseline,* s.l.: Belgium Campus ITversity.

Burak Pak, A. C. A. V. M., 2017. *FixMyStreet Brussels: Socio-Demographic Inequality in Crowdsourced Civic Participation*. [Online]
Available at: https://arxiv.org/abs/1708.02274
[Accessed 25 09 2026].

CivicPlus, n.d.. *Overview, civicplus.com fact-sheet*. [Online]
Available at: https://www.civicplus.com/seeclickfix-311-crm/
[Accessed 25 09 2026].

Peter Matthews, A. R. E. N. A. P., 2018. *research.mysociety.org, FixMyStreet: The geography of citizen reporting*. [Online]
Available at: https://research.mysociety.org/publications/fixmystreet-geography-neighbourhood-issues
[Accessed 25 09 2026].

SeeClickFix, n.d.. *SeeClickFix Case Study: Integration with Cityworks in the City of New Haven*.. [Online]
Available at: https://seeclickfix.com/media/seeslickfix_case_study.pdf
[Accessed 25 09 2026].

CivicPlus (n.d.) *SeeClickFix 311 CRM*. Available at: https://www.civicplus.com/seeclickfix-311-crm/ (Accessed: 25 September 2026). CivicPlus

SeeClickFix (n.d.) *SeeClickFix Case Study: Integration with Cityworks in the City of New Haven*. Available at: https://seeclickfix.com/media/seeslickfix_case_study.pdf (Accessed: 25 September 2026).

React (2026) *Quick Start*. Available at: https://react.dev/learn (Accessed: 25 September 2026). React

Vite (2026) *Getting Started*. Available at: https://vite.dev/guide/ (Accessed: 25 September 2026).

Supabase (2026) *Database*. Available at: https://supabase.com/docs/guides/database/overview (Accessed: 25 September 2026). Supabase

Supabase (2026) *Row Level Security*. Available at: https://supabase.com/docs/guides/database/postgres/row-level-security (Accessed: 25 September 2026). Supabase

Vercel (2026) *Vite on Vercel*. Available at: https://vercel.com/docs/frameworks/frontend/vite (Accessed: 25 September 2026).

Vitest (2026) *Getting Started*. Available at: https://vitest.dev/guide/ (Accessed: 25 September 2026).